หัวข้อโครงงานวิศวกรรมคอมพิวเตอร์ หลักสูตรวิศวกรรมศาสตร์บัณฑิต

สาขาวิชาวิศวกรรมคอมพิวเตอร์

ปีการศึกษา 2567

ชื่อโครงงาน: Effect of quantum random number on meta-heuristic algorithms' initialization

ผลกระทบของการใช้งานเลขสุ่มเชิงควอนตัมกับการสร้างประชากรเริ่มต้นของ อัลกอริทึมแบบเมตาฮิวริสติก

สมาชิกกลุ่ม: 1. นาย บุญฤทธิ์ สำราญ 64070501028 boonyaritsamran@gmail.com

2. นาย กฤษฎ์ คล่องวิถี 64070501059 ArtkritSk138@gmail.com

ที่ปรึกษา: ผศ. ราชวิชช์ สโรชวิกสิต

ที่ปรึกษาร่วม: ดร. ประพงษ์ ปรีชาประพาฬวงศ์

ผศ.ดร. รุจิภาส บวรทวีปัญญา

By signing this, I hereby acknowledge that I have read the final report and approved this attached draft.

Advisor name: _____

By date: 11/5/2568

FOR ADVISOR

EFFECT OF QUANTUM RANDOM NUMBER ON META-HEURISTIC ALGORITHMS' INITIALIZATION

MR. BOONYARIT SAMRAN
MR. KRIT KLONGWITHEE

A PROJECT SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR
THE DEGREE OF BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)
FACULTY OF ENGINEERING
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY THONBURI
2024

Effect of quantum random number on meta-heuristic algorithms' initialization

Mr. Boonyarit Samran
Mr. Krit Klongwithee

A Project Submitted in Partial Fulfillment
of the Requirements for
the Degree of Bachelor of Engineering (Computer Engineering)
Faculty of Engineering
King Mongkut's University of Technology Thonburi
2024

Project Committee

. . . . . Project Advisor
(Asst.Prof. Rajchawit Sarochawikasit)

. . . . . Project Co-Advisor
(Dr. Prapong Prechaprapranwong, Ph.D.)

. . . . . Committee Member
(Asst.Prof. Sanan Srakaew)

. . . . . Committee Member
(Dr. Unchalisa Taetragool, Ph.D.)

Copyright reserved

Project Title	Effect of quantum random number on meta-heuristic algorithms' initialization
Credits	3
Member(s)	Mr. Boonyarit Samran
	Mr. Krit Klongwithee
Project Advisor	Asst.Prof. Rajchawit Sarochawikasit
Co-advisor	Dr. Prapong Prechaprapranwong, Ph.D.
	Assoc.Prof.Dr. Rujipas Bavontaweepanya, Ph.D.
Program	Bachelor of Engineering
Field of Study	Computer Engineering
Department	Computer Engineering
Faculty	Engineering
Academic Year	2024

Abstract

Currently, meta-heuristic algorithms play a vital role in solving complex optimization problems. Many new algorithms, inspired by natural behaviors or mathematical principles, have been developed to improve optimization processes and achieve faster convergence. However, the crucial initialization step, which can greatly influence optimization outcomes, has received relatively little research attention. Although initialization schemes can enhance convergence speed or solution quality by starting near optimal solutions, they typically rely on synthetic entropy sources rather than true natural randomness.

This research focuses on the impact of using quantum truly random numbers in the initialization phase of three meta-heuristic algorithms: Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and Artificial Bee Colony (ABC), compared to the traditional alternative of pseudo-random numbers. We employ quantum random number generators, implemented via quantum circuits and quantum experiments, which derive entropy from fundamental quantum phenomena. Our findings reveal that quantum random numbers exhibit weaker statistical randomness according to standard randomness tests (NIST statistical test suite), compared to high-quality pseudo-random numbers that are specifically designed to pass such tests. However, the Friedman test indicates that using the quantum random numbers in the Genetic Algorithm's initialization has a significant improvement when solving the CEC2021 benchmark problems (Congress on Evolutionary Computation 2021), compared to initialization with pseudo-random numbers. In contrast, for the other two algorithms—Particle Swarm Optimization and Artificial Bee Colony—the application of quantum random numbers does not lead to a statistically significant improvement in optimization performance.

Keywords: Initialization process / Meta-heuristic algorithm / Pseudo-random number generator / Quantum random number generator

หัวข้อปริญญานิพนธ์	ผลกระทบของการใช้งานเลขสุ่มเชิงควอนตัมกับการสร้างประชากรเริ่มต้นของอัลกอริทึมแบบเมตาฮิวริสติก
หน่วยกิต	3
ผู้เขียน	นายบุญฤทธิ์ สำราญ นายกฤษฎ์ คล่องวิถี
อาจารย์ที่ปรึกษา	ผศ. ราชวิชช์ สโรชวิกสิต ดร. ประพงษ์ ปรีชาประพาฬวงศ์ ผศ.ดร. รุจิภาส บวรทวีปัญญา
หลักสูตร	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา	วิศวกรรมคอมพิวเตอร์
ภาควิชา	วิศวกรรมคอมพิวเตอร์
คณะ	วิศวกรรมศาสตร์
ปีการศึกษา	2567

บทคัดย่อ

อัลกอริทึมแบบเมตาฮิวริสติกได้รับความนิยมอย่างแพร่หลายเพื่อแก้ไขปัญหาการหาค่าเหมาะสมที่สุด โดยมีการพัฒนาอัลกอริทึมแบบเมตาฮิวริสติกใหม่จำนวนมากที่ได้แรงบันดาลใจจากพฤติกรรมทางธรรมชาติหรือแนวคิดทางคณิตศาสตร์ เพื่อเพิ่มประสิทธิภาพในการหาคำตอบที่ดีขึ้นและลดเวลาในการลู่เข้าสู่คำตอบ อย่างไรก็ตาม การสร้างประชากรเริ่มต้นที่มีความสำคัญต่อประสิทธิภาพของการหาคำตอบมีความนิยมในการศึกษาจำนวนน้อย แม้ว่าวิธีการสร้างประชากรเริ่มต้นจะสามารถให้การลู่เข้าสู่คำตอบรวดเร็วขึ้นหรือเพิ่มคุณภาพของคำตอบได้ ปัจจุบัน การสร้างประชากรเริ่มต้นส่วนใหญ่ใช้แหล่งกำเนิดความสุ่มที่ถูกสังเคราะห์ขึ้น แทนการใช้แหล่งกำเนิดความสุ่มในธรรมชาติอย่างแท้จริง ซึ่งขัดแย้งกับแนวความคิดพื้นฐานของอัลกอริทึมแบบเมตาฮิวริสติกที่พยายามเลียนแบบธรรมชาติ

การศึกษาครั้งนี้จึงมุ่งเน้นการศึกษาผลกระทบของการใช้เลขสุ่มจริงเชิงควอนตัมในกระบวนการสร้างประชากรเริ่มต้นของอัลกอริทึมแบบเมตาฮิริสติก 3 ชนิด ได้แก่ Genetic Algorithm (GA), Particle Swarm Optimization (PSO) และ Artificial Bee Colony (ABC) เปรียบเทียบกับการใช้เลขสุ่มเทียมจากเครื่องกำเนิดเลขสุ่มเทียมที่นิยมใช้งานทั่วไป โดยเครื่องกำเนิดเลขสุ่มจริงเชิงควอนตัมที่นำมาใช้ในงานวิจัยนี้มีแหล่งที่มาจากวงจรควอนตัม และการทดลองแสงเชิงควอนตัมที่ถูกพัฒนามาจากปรากฏการณ์ธรรมชาติ โดยผลการประเมินคุณสมบัติความสุ่มของตัวเลขด้วยชุดทดสอบ NIST statistical test suite พบว่าเลขสุ่มจริงเชิงควอนตัมมีคุณสมบัติความสุ่มที่ไม่ดีเทียบเท่าเลขสุ่มที่เกิดจากเครื่องกำเนิดเลขสุ่มเทียมที่ถูกออกแบบมาเพื่อหลีกเลี่ยงความไม่สุ่ม และจากการทดสอบด้วยการทดสอบฟรีดแมน (Friedman test) บ่งชี้ให้เห็นว่าการใช้เลขสุ่มจริงเชิงควอนตัมในการสร้างประชากรเริ่มต้นของ Genetic algorithm ส่งผลให้อัลกอริทึมแบบเมตาฮิวริสติกได้ผลลัพธ์แตกต่างการใช้เลขสุ่มเทียมอย่างมีนัยสำคัญ และมีผลลัพธ์ที่ดีในการแก้ปัญหา CEC2021 (Congress on Evolutionary Computation 2021) ในทางตรงกันข้าม สำหรับอัลกอริทึมแบบเมตาฮิวริสติกอีกสองชนิด ได้แก่ Particle Swarm Optimization และ Artificial Bee Colony การใช้เลขสุ่มจริงเชิงควอนตัมไม่ส่งผลให้ประสิทธิภาพการหาคำตอบดีขึ้นอย่างมีนัยสำคัญเมื่อเทียบกับการใช้เลขสุ่มเทียม

คำสำคัญ: กระบวนการสร้างประชากรเริ่มต้น / อัลกอริทึมแบบเมตาฮิวริสติก / เครื่องกำเนิดเลขสุ่มเทียม / เครื่องกำเนิดเลขสุ่มเชิงควอนตัม

กิตติกรรมประกาศ

การทำโครงงานครั้งนี้สำเร็จได้ด้วยความช่วยเหลืออย่างดียิ่งของ ผศ.ราชวิชช์ สโรชวิกสิต ที่ปรึกษาโครงงานที่กรุณาสละเวลาให้คำปรึกษา แนวคิด และข้อเสนอแนะที่เป็นประโยชน์ รวมถึงความรู้ระเบียบวิธีของการเป็นนักวิจัย และความใส่ใจ ติดตามดูแลเอาใจใส่ต่อการทำโครงงานฉบับนี้จนสำเร็จลุล่วงลงได้ดี ผู้จัดทำโครงงานจึงขอกราบขอบพระคุณเป็นอย่างสูงมา ณ ที่นี้

ขอขอบคุณ ผศ.ดร.รุจิภาส บวรทวีปัญญา และ ผศ.ดร.เอกรัฐ พงษ์โอภาส จากมหาวิทยาลัยธรรมศาสตร์ ที่ให้ความอนุเคราะห์ข้อมูลเลขสุ่มควอนตัมที่ได้จากการทดลอง และความรู้ด้านฟิสิกส์ที่เป็นประโยชน์ต่อโครงงานนี้

ขอขอบคุณ ดร.ประพงษ์ ปรีชาประพาฬวงศ์ ในการสังเกตสิ่งที่น่าสนใจจากผลการทดลองความสุ่ม และการให้คำปรึกษาในขั้นตอนการวิจัย

ขอขอบคุณ ภาควิชาวิศวกรรมคอมพิวเตอร์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี ที่ให้การสนับสนุนนักศึกษาของภาควิชา ในการเข้าร่วมกิจกรรมอันเป็นประโยชน์นี้

ท้ายที่สุดนี้ โครงงานนี้จะไม่สามารถสำเร็จได้ หากขาดการสนับสนุนจาก บิดา มารดา รวมทั้งเพื่อน และรุ่นน้องในภาควิชาที่เกี่ยวข้องที่คอยเป็นกำลังใจสำคัญเสมอมา

สารบัญ

หน้า

ABSTRACT ii

บทคัดย่อ iii

กิตติกรรมประกาศ iv

สารบัญ v

สารบัญตาราง vii

สารบัญรูปภาพ viii

สารบัญคำศัพท์ทางเทคนิคและคำย่อ ix

บทที่ 1 บทนำ 1

1.1 ที่มาและความสำคัญ 1

1.2 วัตถุประสงค์ 1

1.3 สมมติฐานการวิจัย 1

1.4 ขอบเขตของโครงงาน 2

1.5 ประโยชน์ที่คาดว่าจะได้รับ 2

1.6 ตารางการดำเนินงาน 2

บทที่ 2 ทฤษฎีความรู้และงานที่เกี่ยวข้อง 4

2.1 อัลกอริทึมแบบเมตาฮิวริสติก (Meta-heuristic Algorithm) 4

2.1.1 การหาค่าเหมาะสมที่สุดแบบกลุ่มอนุภาค (PSO) 4

2.1.2 อัลกอริทึมเชิงพันธุกรรม (GA) 5

2.1.3 อัลกอริทึมแบบอาณานิคมผึ้งเทียม (ABC) 5

2.2 เครื่องกำเนิดเลขสุ่ม (Random Number Generator: RNG) 6

2.2.1 เครื่องกำเนิดเลขสุ่มเทียม (Pseudo-random Number Generator: PRNG) 6

2.2.2 เครื่องกำเนิดเลขสุ่มจริง (True random Number Generator: TRNG) 7

2.3 ปัญหาการหาค่าที่เหมาะสมที่สุด 8

2.3.1 การทดสอบด้วยฟังก์ชันคณิตศาสตร์ 8

2.3.2 ปัญหาการเดินทางของพนักงานขาย (TSP) 9

2.3.3 ปัญหาการหาจุดตัดที่มากที่สุด (ปัญหา Max-cut) 9

2.4 ชุดเครื่องมือทดสอบการเป็นเลขสุ่มจริง (Test suite) 9

2.4.1 NIST STS (National Institute of Standards and Technology Statistical Test Suite) 10

2.5 เครื่องมือสำหรับวิเคราะห์ผลการทดลอง (Experiment analytic tools) 12

2.6 ผลกระทบของการเปลี่ยนแปลงวิธีการสร้างประชากรเริ่มต้นต่อการทำงานของอัลกอริทึมแบบเมตาฮิวริสติก 12

บทที่ 3 วิธีการดำเนินงาน 13

3.1 ภาพรวมของการดำเนินงาน 13

3.2 การพัฒนาเครื่องกำเนิดเลขสุ่ม 14

3.3 การทดลอง 20

3.3.1 การสร้างประชากรเริ่มต้นจากเลขสุ่มสำหรับปัญหาแต่ละประเภท 20

3.3.2 การวัดคุณภาพประชากร 21

3.4 การวิเคราะห์ผลที่ได้จากการทดลอง 22

3.4.1 การวิเคราะห์การชนะของค่าเหมาะสมโดยรวม และความหลากหลายของประชากร 22

3.4.2 การวิเคราะห์ค่าสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปร (Pearson correlation coefficient analysis) 23

3.4.3 การวิเคราะห์ผลกระทบของเครื่องกำเนิดเลขสุ่มและคุณภาพประชากรด้วยการทดสอบฟรีดแมน (Friedman test) 24

บทที่ 4 ผลการดำเนินงาน **25**
4.1 ผลการทดสอบคุณสมบัติความสุ่มสุ่มของเครื่องกำเนิดเลขสุ่ม 25
4.2 ผลการทดลองการทำงานของอัลกอริทึมแบบเมตาฮิวริสติก 28
4.2.1 ผลการวิเคราะห์การชนะของค่าเหมาะสมโดยรวม และความหลากหลายของประชากร 28
4.2.2 ผลการวิเคราะห์ค่าสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปร 29
4.2.3 ผลการวิเคราะห์ผลกระทบของเครื่องกำเนิดเลขสุ่มและคุณภาพประชากรด้วยการทดสอบฟรีดแมน 31
4.2.4 การอภิปรายผลการทดลองต่อสมมติฐานการวิจัย 33

บทที่ 5 บทสรุป **35**
5.1 สรุปผลการดำเนินงาน 35
5.2 ปัญหาที่พบและการแก้ไข 35
5.2.1 ข้อจำกัดของอุปกรณ์การทดลอง 35
5.2.2 ปัญหาการอ่านไฟล์ผิดพลาด 35
5.2.3 ปัญหาความไม่แน่นอนของผลการทดลอง 35
5.3 ข้อจำกัดและข้อเสนอแนะ 36

หนังสืออ้างอิง **37**

สารบัญตาราง

ตารางที่ **หน้า**

3.1 ตัวอย่างข้อมูลและผลลัพธ์จากการจัดการข้อมูลก่อนการทดสอบฟรีดแมน 24
4.1 ตารางเปรียบจำนวนค่า p-value of p-value ในช่วงต่าง ๆ จากการทดสอบด้วย NIST STS 26
4.2 ตารางเปรียบเทียบจำนวนค่า pass ratio ในช่วงต่าง ๆ จากการทดสอบด้วย NIST STS 28
4.3 ตารางค่า p-value ของการทดสอบฟรีดแมนจำแนกตามชนิดของอัลกอริทึม และปัญหาที่ใช้ทดสอบ 33

สารบัญรูป

รูปที่ **หน้า**

1.1 ตารางแสดงตารางการดำเนินงานภาคการศึกษาที่ 1 2
1.2 ตารางแสดงตารางการดำเนินงานภาคการศึกษาที่ 2 3
2.1 ลำดับการทำงานของอัลกอริทึมเชิงพันธุกรรม 5
2.2 ลำดับการทำงานของอัลกอริทึมแบบอาณานิคมผึ้งเทียม 5
2.3 การเปรียบเทียบเลขสุ่มขนาด 2 มิติระหว่างเลขสุ่มเทียม และ เลขสุ่ม Quasirandom 7
2.4 วงจรเลขสุ่มเชิงควอนตัมขนาด 1 คิวบิต 8
2.5 การผลิตเลขสุ่มเชิงควอนตัมจากการทดลองจำแนกลำแสง 8
3.1 กระบวนการทดลอง 13
3.2 ขั้นตอนของการเครื่องกำเนิดเลขสุ่มจริงจากการทดลองแสงเชิงควอนตัม 14
3.3 ข้อมูลจากการทดลอง และปัญหาความสกปรกของข้อมูล 15
3.4 state machine diagram ของการทำความสะอาดข้อมูล 15
3.5 ลักษณะของข้อมูลที่ถูกการทำความสะอาด 15
3.6 การสร้างตารางจำแนกสำหรับเลขสุ่มขนาด 3 บิต 16
3.7 การใช้ตารางจำแนกเพื่อสร้างเลขสุ่มขนาด 3 บิตต่อข้อมูล Dacay time 1 ค่า 16
3.8 วงจรควอนตัมคอมพิวเตอร์สำหรับการสร้างเลขสุ่ม 17
3.9 การสร้างเลขสุ่มของอัลกอริทึม Quasirandom ที่ใช้ในการทดลอง 18
3.10 การสร้างเลขสุ่มของอัลกอริทึม Quasirandom ที่ใช้ในการทดลอง (ต่อ) 18
3.11 การสร้างเลขสุ่มเทียมจากอัลกอริทึม MT19937 ที่ใช้ในการทดลอง 20
3.12 ภาพกระบวนการทำความสะอาด การวิเคราะห์ข้อมูล และตัวอย่างแผนภาพที่เกิดขึ้นจากการทดลอง 23
4.1 ค่า p-value of p-value ที่ได้จากการทดสอบความสุ่มด้วย NIST STS 26
4.2 ค่า pass ratio ที่ได้จากการทดสอบความสุ่มด้วย NIST STS 27
4.3 แผนภาพความร้อนแสดงจำนวนการชนะของค่าที่เกิดขึ้นจากการสร้างประชากรเริ่มต้นด้วยเครื่องกำเนิดเลขสุ่มชนิดต่าง ๆ 29
4.4 แผนภาพความร้อนแสดงจำนวนการชนะของค่าที่เกิดขึ้นจากการสร้างประชากรเริ่มต้นด้วยเครื่องกำเนิดเลขสุ่มประเภทต่าง ๆ 29
4.5 แผนภาพความร้อนแสดงสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปรของอัลกอริทึมแบบเมตาฮิวริสติกที่ใช้แก้ไขปัญหา CEC2021 30
4.6 แผนภาพความร้อนแสดงสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปรของอัลกอริทึมแบบเมตาฮิวริสติกที่ใช้แก้ไขปัญหา TSP 30
4.7 แผนภาพความร้อนแสดงสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปรของอัลกอริทึมแบบเมตาฮิวริสติกที่ใช้แก้ไขปัญหา Max-cut 31
4.8 แผนภาพความร้อนแสดงลำดับเฉลี่ยของประชากรสุดท้ายจากเครื่องกำเนิดเลขสุ่มชนิดต่าง ๆ สำหรับแก้ไขปัญหา CEC2021 32
4.9 แผนภาพความร้อนแสดงลำดับเฉลี่ยของประชากรสุดท้ายจากเครื่องกำเนิดเลขสุ่มชนิดต่าง ๆ สำหรับแก้ไขปัญหา TSP 32
4.10 แผนภาพความร้อนแสดงลำดับเฉลี่ยของประชากรสุดท้ายจากเครื่องกำเนิดเลขสุ่มชนิดต่าง ๆ สำหรับแก้ไขปัญหา Max-cut 33

สารบัญคำศัพท์ทางเทคนิคและคำย่อ

ABC	=	Artificial Bee Colony, อัลกอริทึมอาณานิคมผึ้งเทียม
CEC2021	=	Congress on Evolutionary Computation 2021, การแข่งขันการแก้ไขปัญหาค่าที่เหมาะสมที่สุดในรูปแบบฟังก์ชันคณิตศาสตร์
GA	=	Genetic Algorithm, อัลกอริทึมเชิงพันธุกรรม
Max-cut	=	ปัญหาการหาผลรวมค่าน้ำหนักที่มากที่สุดของเส้นที่ลากตัดกราฟเป็นสองฝั่ง
NIST STS	=	National Institute of Standards and Technology Statistical Test Suite, ชุดเครื่องมือทดสอบความสุ่มของเลขสุ่ม
OPFUNU	=	OPtimization benchmark FUnctions in NUmpy
PRNG	=	Pseudo-random Number Generator, เครื่องกำเนิดเลขสุ่มเทียม
PSO	=	Particle Swarm Optimization, อัลกอริทึมการหาค่าเหมาะสมที่สุดแบบกลุ่มอนุภาค
QRNG	=	Quantum Random Number Generator, เครื่องกำเนิดเลขสุ่มควอนตัม
TSP	=	Traveling Salemans Problem, ปัญหาการเดินทางของพนักงานขาย

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

ตั้งแต่ช่วงปี ค.ศ. 1950 ที่แนวคิดอัลกอริทึมแบบเมตาฮิวริสติก ได้ถูกนำเสนอครั้งแรกในรูปแบบวิธีการหาค่าเหมาะสมที่สุด [1] จนถึงปัจจุบัน อัลกอริทึมแบบเมตาฮิวริสติก ยังคงมีความนิยมถูกใช้เป็นเครื่องมือในการแก้ไขปัญหาประเภทการหาค่าเหมาะสมที่สุด (Optimization) เช่น การหาระยะทางที่เหมาะสมที่สุดบนแผนที่ การลดจำนวนตัวแปรในการคำนวณ และปัญหาทางด้านวิศวกรรมอื่น ๆ ทำให้การศึกษาอัลกอริทึมแบบเมตาฮิวริสติกเกิดขึ้นใหม่อย่างต่อเนื่อง ทั้งในด้านการออกแบบอัลกอริทึมรูปแบบใหม่เพื่อประสิทธิภาพที่ดีมากขึ้น หรือการประยุกต์อัลกอริทึมแบบเมตาฮิวริสติกให้สามารถแก้ไขปัญหารูปแบบใหม่ได้

ในเวลาใกล้เคียงกับการพัฒนาอัลกอริทึมแบบเมตาฮิวริสติก แนวคิดการคำนวณเชิงควอนตัมก็ได้ถูกนำเสนอขึ้นเพื่อที่จะแก้ไขปัญหาที่คอมพิวเตอร์แบบทั่วไปแก้ไขปัญหาได้ช้า นำไปสู่การสร้างควอนตัมคอมพิวเตอร์สำเร็จครั้งแรกในปี ค.ศ. 2000 และเกิดนวัตกรรมต่าง ๆ ที่อาศัยหลักการเชิงควอนตัม เช่น เครื่องมือทางการแพทย์ อุปกรณ์อิเล็กทรอนิกซ์ อัลกอริทึมการเข้ารหัสลับ และอัลกอริทึมการหาค่าเหมาะสมที่สุดที่อาศัยหลักการควอนตัม เช่น QAOA (Quantum Approximate Optimization Algorithm) เป็นต้น

เนื่องจาก มีการศึกษาจำนวนมากได้ศึกษาผลกระทบการเปลี่ยนแปลงกระบวนการภายในอัลกอริทึมแบบเมตาฮิวริสติก เช่น การคิดค้นการคัดเลือกโครโมโซมรูปแบบใหม่บนอัลกอริทึม GA (Genetic Algorithm) [2], [3] การปรับปรุงกระบวนการการสร้างคำตอบใหม่ในอัลกอริทึม PSO (Particle Swarm Optimization) ให้มีประสิทธิภาพดีมากขึ้น [4] และการเปลี่ยนวิธีการสร้างประชากรเริ่มต้น (Initialization) ให้เป็นการแจกแจงรูปแบบต่าง ๆ บนอัลกอริทึมแบบเมตาฮิวริสติก [5] ทำให้ผู้จัดทำเกิดความสงสัยถึงปัจจัยที่ส่งผลต่อการเรียนรู้ของอัลกอริทึมแบบเมตาฮิวริสติก โดยเฉพาะผลกระทบการเปลี่ยนแปลงวิธีการสร้างประชากรเริ่มต้นต่อคุณภาพคำตอบ หรือการทำงานของอัลกอริทึมแบบเมตาฮิวริสติก ซึ่งวิธีการสร้างประชากรเริ่มต้นของอัลกอริทึมแบบเมตาฮิวริสติกส่วนใหญ่มีการสร้างประชากรเริ่มต้นจากเครื่องกำเนิดเลขสุ่มเทียม (PRNG, Pseudo-random Number Generator) เนื่องจากเป็นเลขสุ่มที่สามารถคำนวณได้ง่ายโดยหน่วยประมวลผลกลาง แต่จะมีข้อจำกัดด้านรูปแบบของเลขสุ่มเทียมที่ถูกสร้างขึ้นจากสมการสามารถเกิดรูปแบบซ้ำได้เมื่อครบคาบการสุ่ม ขัดแย้งต่อแนวคิดของอัลกอริทึมแบบเมตาฮิวริสติก บางชนิดที่ออกแบบเพื่อเลียนแบบธรรมชาติ เช่น GA, PSO, และ ABC (Artificial Bee Colony) ดังนั้น คณะผู้จัดทำจึงสนใจที่จะศึกษาผลกระทบของการสร้างประชากรเริ่มต้นด้วยเลขสุ่มจริงที่ได้จากเครื่องกำเนิดเลขสุ่มควอนตัม (QRNG, Quantum Random Number Generator) ต่อการเรียนรู้ของอัลกอริทึมแบบเมตาฮิวริสติกที่เลียนแบบธรรมชาติ เปรียบเทียบกับผลกระทบของการใช้เครื่องกำเนิดเลขสุ่มเทียม ในการสร้างประชากรเริ่มต้นทำงานในอัลกอรีทึมชนิดเดียวกัน โดยมีขอบเขตที่จะศึกษาอัลกอริทึมแบบเมตาฮิวริสติก 3 ชนิด ได้แก่ GA, PSO, และ ABC และปัญหาที่นำมาทดสอบทั้งหมด 3 ปัญหา ได้แก่ TSP (Traveling Salesman Problem), ปัญหา Max-cut (ปัญหาการหาจุดตัดที่มากที่สุด), และ ปัญหา CEC2021 หรือ ปัญหาการหาค่าเหมาะสมที่สุดผ่านจากปัญหาการแข่งขัน Congress on Evolutionary Computation ปี 2021

1.2 วัตถุประสงค์

1. เพื่อศึกษาผลกระทบของการใช้เลขสุ่มจริงเชิงควอนตัมจาก QRNG ต่อการทำงานของอัลกอริทึมต่าง ๆ ของคอมพิวเตอร์ ซึ่งอาจเป็นแนวทางพัฒนาประสิทธิภาพการแก้ไขปัญหาการหาค่าเหมาะสมที่สุด

2. เพื่อวัดประสิทธิภาพของเลขสุ่มจริงจากแหล่งที่มาต่าง ๆ ผ่านชุดการทดสอบ (Test suite) และอภิปรายผลกระทบของคุณสมบัติการเป็นเลขสุ่มควอนตัมกับปัญหาที่เกี่ยวข้องกับอัลกอริทึมแบบเมตาฮิวริสติก

1.3 สมมติฐานการวิจัย

1. QRNG สามารสร้างเลขสุ่มที่มีคุณสมบัติความสุ่มที่ดีกว่า PRNG

2. ความหลากหลายของประชากรเริ่มต้นส่งผลต่อค่าเหมาะสมของประชากรรุ่นสุดท้าย

3. ประชากรเริ่มต้นที่ได้จาก QRNG มีความหลากหลายมากกว่าประชากรเริ่มต้นที่ได้จาก PRNG

4. ประชากรที่ได้จาก QRNG มีค่าเหมาะสมใกล้ค่าเหมาะสมที่สุดมากกว่าประชากรที่ได้จาก PRNG เมื่อพิจารณา ณ รอบสุดท้ายของการทำงานขออัลกอริทึมแบบเมตาฮิวริสติก

5. การเลือกใช้ QRNG ในการสร้างประชากรเริ่มต้น สามารถทำให้คุณภาพประชากรสุดท้ายสุดท้ายแตกต่าง และใกล้ค่าเหมาะสมที่สุดมากกว่า เมื่อเปรียบเทียบกับประชากรเริ่มต้นที่ได้จาก PRNG

1.4 ขอบเขตของโครงงาน

1. วิเคราะห์คุณภาพเลขสุ่มรูปแบบ Quasirandom, เลขสุ่มจากอัลกอริทึม MT19937, เลขสุ่มจาก IBM ควอนตัมคอมพิวเตอร์ และเลขสุ่มที่ได้จากการทดลองแสงเชิงควอนตัม

2. เปรียบเทียบผลกระทบของการสร้างประชากรเริ่มต้นโดยใช้เลขสุ่มชนิดต่าง ๆ บนอัลกอริทึมแบบเมตาฮิวริสติก โดยมีรายละเอียดดังนี้

    - อัลกอริทึมแบบเมตาฮิวริสติก 3 ชนิด ที่ใช้ในการศึกษาถูกจัดอยู่ในชนิดเลียนแบบธรรมชาติ (Nature-inspired meta-heuristic algorithm) ประกอบไปด้วย GA, PSO และ ABC
    - ปัญหาที่ใช้ทดสอบอัลกอริทึมแบบเมตาฮิวริสติกมีทั้งหมด 3 ปัญหา ได้แก่ ปัญหา TSP, ปัญหา Max-cut, และ ปัญหา CEC2021

1.5 ประโยชน์ที่คาดว่าจะได้รับ

1. ระบุผลกระทบ หรือความแตกต่างระหว่างการใช้เลขสุ่มจาก QRNG และ PRNG เพื่อสร้างประชากรเริ่มต้นของอัลกอริทึมแบบเมตาฮิวริสติกด้วยการพิจารณาค่าเหมาะสมโดยรวม (Global fitness) และความหลากหลายของประชากร (Population diversity)

2. ระบุความสัมพันธ์ระหว่างประสิทธิภาพการทำงานของอัลกอริทึมแบบเมตาฮิวริสติก ปัญหาที่ใช้ทดสอบ และคุณภาพของเลขสุ่มที่ได้จากเครื่องกำเนิดเลขสุ่มแต่ละชนิด

1.6 ตารางการดำเนินงาน

เนื้อหา	เนื้อหางานย่อย	เวลา (เดือน และสัปดาห์)																			
		สิงหาคม 2567				กันยายน 2567				ตุลาคม 2567				พฤศจิกายน 2567				ธันวาคม 2567			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1. การศึกษาหัวข้อโครงงาน	1.1. ตั้งหัวข้อโครงงาน และปัญหาที่เกี่ยวข้อง																				
	1.2. กำหนดจุดประสงค์ของโครงงาน																				
	1.3. ศึกษาเอกสาร และความรู้ที่เกี่ยวข้อง																				
	1.4. กำหนดขอบเขตการศึกษา และแนวคิดการทดลอง																				
	1.5. ตั้งสมมติฐานของโครงงาน																				
2. การจัดเตรียมเอกสารที่เกี่ยวข้องกับโครงงาน	2.1. ทบทวนวรรณกรรม																				
	2.2. จัดทำข้อเสนอโครงงาน																				
	2.3. ซักซ้อม และนำเสนอข้อเสนอโครงงาน																				
3 การพัฒนาคำสั่งเครื่องกำเนิดเลขสุ่ม	3.1. พัฒนาคำสั่งเครื่องกำเนิดเลขสุ่มเทียม																				
	3.2. พัฒนาคำสั่งเครื่องกำเนิดเลขสุ่มจริง																				
	3.3. ตรวจวัดประสิทธิภาพของเลขสุ่มที่ได้รับ																				
	3.4. ออกแบบ และพัฒนาคำสั่งในการแปลผลเลขสุ่มให้กลายเป็นวิธีการสร้างประชากรเริ่มต้น																				
4. การวิเคราะห์ และสรุปผลการทดลอง	4.1. เปรียบเทียบประสิทธิภาพของเลขสุ่มทั้งหมด																				
	4.2. จัดทำเอกสารนำเสนอความคืบหน้า																				

รูปที่ 1.1 ตารางแสดงตารางการดำเนินงานภาคการศึกษาที่ 1

เนื้อหา	เนื้อหางานย่อย	เวลา (เดือน และสัปดาห์)																			
		มกราคม 2568				กุมภาพันธ์ 2568				มีนาคม 2568				เมษายน 2568				พฤษภาคม 2568			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1. การทดลอง	1.1. ปรับปรุงคำสั่งให้เหมาะสมกับเป้าหมายการทดลอง																				
	1.2. ทดลองใช้เลขสุ่มรูปแบบต่าง ๆ กับอัลกอริทึมแบบเมตาฮิวริสติกเพื่อแก้ปัญหาที่กำหนด																				
	1.3. เปรียบเทียบประสิทธิภาพการทำงานอัลกอริทึมแบบเมตาฮิวริสติก ระหว่างการใช้เลขสุ่มจริง และเลขสุ่มเทียม ในขั้นตอนการ initialization																				
2. การวิเคราะห์ และสรุปผลการทดลอง	2.1. พิสูจน์สมมติฐานด้วยค่าทางสถิติ																				
	2.2. สร้างแผนภาพประกอบคำอธิบาย																				
	2.3. สรุปผลการทดลองทั้งหมด																				
3 การจัดเตรียมการนำเสนอโครงงาน	3.1. จัดทำเอกสารนำเสนอโครงงาน																				
	3.2. ซักซ้อมการนำเสนอโครงงาน																				
	3.3. นำเสนอโครงงาน																				

รูปที่ 1.2 ตารางแสดงตารางการดำเนินงานภาคการศึกษาที่ 2

บทที่ 2 ทฤษฎีความรู้และงานที่เกี่ยวข้อง

2.1 อัลกอริทึมแบบเมตาฮิวริสติก (Meta-heuristic Algorithm)

อัลกอริทึมแบบเมตาฮิวริสติก [6] คือ แนวคิดการพัฒนาโปรแกรมโดยใช้หลักการทดลองและปรับปรุงซ้ำ (Trial and error) เพื่อค้นหาคำตอบ หรือประชากรที่เหมาะสมกับปัญหาที่มีความซับซ้อน เช่น การค้นหาจุดต่ำสุดสัมบูรณ์จากสมการที่มีจุดต่ำสุดสัมพัทธ์จำนวนมาก การค้นหาเส้นทางขนส่งสินค้าที่ทำให้เกิดมลพิษน้อยที่สุด [7] เป็นต้น ตัวอย่าง อัลกอริทึมแบบเมตาฮิวริสติกที่สำคัญ เช่น GA, PSO, Differential Evolution (DE) เป็นต้น ซึ่งการทำงานของอัลกอริทึมแบบเมตาฮิวริสติก จะถูกแบ่งเป็น 2 ประเภท เมื่อพิจารณาจากจำนวนประชากรที่ถูกใช้ค้นหา 1 ครั้ง [8] ได้แก่ Population-based และ Single solution search โดยมีรายละเอียด ดังนี้

- อัลกอริทึมแบบเมตาฮิวริสติกประเภท Single-solution คือ อัลกอริทึมแบบเมตาฮิวริสติกที่ทำกระบวนการแก้ไขคำตอบให้ดีขึ้นครั้งละ 1 คำตอบ เช่น SA (Simulated annealing), Iterated local search และ Variable neighborhood search
- อัลกอริทึมแบบเมตาฮิวริสติกประเภท Population-based meta-heuristic คือ อัลกอริทึมแบบเมตาฮิวริสติก ที่ทำกระบวนการแก้ไขคำตอบให้ดีขึ้นครั้งละหลายคำตอบ เช่น อัลกอริทึมกลุ่มวิวัฒนาการ (Evolutionary algorithm) และอัลกอริทึมความฉลาดแบบกลุ่ม (Swarm intelligence algorithm)

สำหรับการศึกษาครั้งนี้ คณะผู้จัดทำได้ตัดสินใจเลือกอัลกอริทึมแบบเมตาฮิวริสติกเลียนแบบธรรมชาติ ทั้งหมด 3 ชนิด เพื่อเปรียบเทียบประสิทธิภาพการทำงาน โดยคำนึงถึงความสะดวกในการค้นหางานศึกษาในอดีต [9] และความนิยมของการอ้างอิงอัลกอริทึมในปี ค.ศ. 2022 ได้แก่ PSO, GA และ ABC ซึ่งแต่ละอัลกอริทึมมีรายละเอียด ดังนี้

2.1.1 การหาค่าเหมาะสมที่สุดแบบกลุ่มอนุภาค (PSO)

การหาค่าเหมาะสมที่สุดแบบกลุ่มอนุภาคถูกนำเสนอครั้งแรกในปี ค.ศ. 1995 [10] เป็นอัลกอริทึมที่ได้รับแรงบันดาลใจจากพฤติกรรมของสัตว์ที่หาอาหารร่วมกันเป็นกลุ่ม เช่น นก หรือ ลูกปลา ซึ่งสัตว์แต่ละตัวจะมีการเคลื่อนที่เป็นของตนเอง และสัตว์ที่อยู่บริเวณใกล้แหล่งอาหารจะมีอิทธิพลต่อการเดินทางของสัตว์ตัวอื่น ๆ

การหาค่าเหมาะสมที่สุดแบบกลุ่มอนุภาค [11] จะแทนที่สัตว์แต่ละตัวเป็นอนุภาคจำนวนหนึ่งอนุภาค หรือเป็นคำตอบที่เป็นไปได้ แต่ละอนุภาคจะมีความเร็วในการเลื่อนตำแหน่ง ตำแหน่งปัจจุบัน และตำแหน่งที่ดีที่สุดของอนุภาคนั้น ซึ่งในการทำงานของ PSO นอกจากแต่ละอนุภาคจะเคลื่อนไหวเข้าหาคำตอบที่เหมาะสมด้วยความเร็วของตนเองแล้ว ยังมีส่วนประกอบของความเร็วที่ได้รับอิทธิพลจากความเร็วของกลุ่มด้วย ทำให้กลุ่มอนุภาคเคลื่อนที่ไปในทิศทางเดียวกัน เพื่อค้นหาค่าบริเวณรอบคำตอบที่ดีที่สุด

หลักการทำงานของการหาค่าเหมาะสมที่สุดแบบกลุ่มอนุภาคมีรายละเอียด ดังนี้

$$f(x_1, x_2, x_3, ..., x_n) = f(X) \tag{2.1}$$

กำหนดให้ X คือ ชุดตัวแปรที่ต้องหาคำตอบที่เหมาะสมที่สุดในฟังก์ชันใด ๆ

x_i คือ ตัวแปรในแต่ละมิติ

การค้นหาคำตอบจะเป็นการค้นหาแบบขนาน (Parallel search) ภายใต้ความสัมพันธ์

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad \text{with} \quad x_i^0 \sim U(x_{\min}, x_{\max}) \tag{2.2}$$

โดยที่ x_i^t คือ ตำแหน่งของอนุภาคลำดับ i ณ เวลา t

v_t คือ ความเร็วของอนุภาคลำดับ i ณ เวลา t เกิดจากการรวมกันของความเร็ว ณ ตำแหน่งที่ดีที่สุดของอนุภาคลำดับ i และ ความเร็ว ณ ตำแหน่งที่ดีที่สุดของกลุ่มอนุภาค

$U(x_{min}, x_{max})$ คือ การแจกแจงเอกรูป (Uniform distribution) ภายในช่วงค่า x_{min}, x_{max}

2.1.2 อัลกอริทึมเชิงพันธุกรรม (GA)

อัลกอริทึมเชิงพันธุกรรมเป็นอัลกอริทึมเชิงวิวัฒนาการ (Evolutionary algorithm) [12], [13] ถูกนำเสนอครั้งแรกโดย John Holland ในช่วงปี ค.ศ. 1970 โดยได้รับแรงบันดาลใจจากหลักการการคัดเลือกโดยธรรมชาติ (Natural selection) อาศัยกระบวนการทางชีววิทยาทำให้เกิดการแลกเปลี่ยนสารพันธุกรรมระหว่างประชากร หรือเรียกว่า โครโมโซมของประชากร เพื่อใช้แก้ไขปัญหาที่ซับซ้อนหรือไม่สามารถแก้ไขได้อย่างมีประสิทธิภาพด้วยวิธีการคณิตศาสตร์แบบดั้งเดิม

ขั้นตอนการทำงานของอัลกอริทึมเชิงพันธุกรรม (รูปที่ 2.1) จะเริ่มจากการสุ่มประชากรเริ่มต้น แล้วนำประชากรมาดำเนินการด้วยตัวดำเนินการเชิงพันธุกรรม (Genetic operator) ได้แก่ การคัดเลือก (Selection operator), การผสมชิ้นส่วนพันธุกรรม (Crossover operator) และ การกลายพันธุ์ (Mutation operator) ประชากรในระบบจะเกิดการพัฒนาลักษณะที่ดีผ่านการประเมินคะแนนจากฟังก์ชันวัตถุประสงค์ จนได้ประชากรที่สามารถอยู่ในสภาพแวดล้อมนั้นได้ หรือคำตอบที่เหมาะสมที่สุดภายใต้ข้อจำกัดของปัญหา

```
Input:
   Population Size, n
   Maximum number of iterations, MAX
Output:
   Global best solution, Y_bt
begin
   Generate initial population of n chromosomes Y_i (i = 1,2,....,n)
   Set iteration counter t = 0
   Compute the fitness value of each chromosomes
   while (t < MAX)
      Select a pair of chromosomes from initial population based on fitness
      Apply crossover operation on selected pair with crossover probability
      Apply mutation on the offspring with mutation probability
      Replace old population with newly generated population
      Increment the current iteration t by 1.
   end while
   return the best solution, Y_bt
end
```

รูปที่ 2.1 ลำดับการทำงานของอัลกอริทึมเชิงพันธุกรรม

(ที่มา: [12])

2.1.3 อัลกอริทึมแบบอาณานิคมผึ้งเทียม (ABC)

อัลกอริทึมแบบอาณานิคมผึ้งเทียม [14] เป็นอัลกอริทึมที่ได้รับแรงบันดาลใจจากพฤติกรรมการบินหาอาหารของฝูงผึ้ง ซึ่งผึ้งแต่ละตัวจะมีหน้าที่แตกต่างกัน ได้แก่ ผึ้งงาน (Employed bee) จะเป็นผึ้งที่บินไปยังแหล่งอาหารที่เคยสำรวจแล้ว และคอยรายงานข้อมูลระยะทางและทิศทางให้กับผึ้งตัวอื่น ผึ้งเฝ้าดู (Onlooker bee) เป็นผึ้งที่อาศัยอยู่ในรัง มีหน้าที่ตรวจสอบข้อมูลแหล่งอาหารที่มีอยู่ทั้งหมดกับข้อมูลแหล่งอาหารที่ได้จากผึ้งสำรวจ (Scout bee) ที่มีหน้าที่ในการหาอาหารในพื้นที่ใหม่ที่ผึ้งงานอาจจะไม่เคยบินผ่าน (รูปที่ 2.2)

เนื่องจากอัลกอริทึมแบบอาณานิคมผึ้งเทียมมีความสามารถในการค้นหาคำตอบใหม่บนปริภูมิการค้นหา จึงมีความสามารถในการแก้ไขปัญหาการติดในบริเวณค่าคำตอบที่ดีที่สุดเฉพาะที่ (Local optima) ได้เร็วกว่าเทคนิคการหาค่าเหมาะสมที่สุดแบบกลุ่มอนุภาคกรณีที่ปัญหามีความซับซ้อน [15]

```
Initialization Phase
REPEAT
      Employed Bees Phase
      Onlooker Bees Phase
      Scouts Bees Phase
      Memorize the best solution attained till now
UNTIL (Cycle = Maximum Cycle Number or a Maximum CPU Time)
ABC Algorithm- Pseudocode.
```

รูปที่ 2.2 ลำดับการทำงานของอัลกอริทึมแบบอาณานิคมผึ้งเทียม

(ที่มา: [14])

2.2 เครื่องกำเนิดเลขสุ่ม (Random Number Generator: RNG)

เครื่องกำเนิดเลขสุ่มสามารถแบ่งประเภทตามลักษณะของเลขสุ่มได้ 2 ประเภท ได้แก่ เครื่องกำเนิดเลขสุ่มเทียม และเครื่องกำเนิดเลขสุ่มจริง โดยจะมีรายละเอียด ดังนี้

2.2.1 เครื่องกำเนิดเลขสุ่มเทียม (Pseudo-random Number Generator: PRNG)

เครื่องกำเนิดเลขสุ่มเทียม คือ แบบจำลอง อัลกอริทึม หรือวิธีการที่สามารถสร้างชุดของเลขที่คาดเดารูปแบบได้ยากโดยอาศัยการคำนวณสมการทางคณิตศาสตร์ วิธีการสร้างเลขสุ่มลักษณะนี้จะมีความสามารถในการใช้ seed เพื่อรักษาผลของเลขเลขสุ่มได้ โดยในการศึกษาครั้งนี้ คณะผู้จัดทำจะสนใจเครื่องกำเนิดเลขสุ่มเทียมทั้งหมด 2 ชนิด ได้แก่

- MT19937 (Mersenne Twister 19937)

  Mersenne Twister [16] เป็นอัลกอริทึมการสร้างเลขสุ่มที่ได้รับความนิยมอย่างแพร่หลายในภาษาการเขียนโปรแกรม เช่น Python, R, Ruby และ MATLAB เป็นต้น ชื่อของอัลกอริทึมนี้มาจากการใช้คาบของเลขสุ่ม (Period) ที่มีความยาวสูงมากซึ่งอยู่ในรูปของเลขสุ่ม Mersenne (Mersenne prime, M_p) โดยคำนวณได้จากสมการ

$$M_p = 2^p - 1 \tag{2.3}$$

เมื่อ p คือ จำนวนเฉพาะ

สำหรับอัลกอริทึม Mersenne Twister ที่นิยมใช้อยู่ในปัจจุบัน คือ MT19937 ซึ่งมีคาบการเกิดซ้ำ (Period) ยาวถึง $2^{19937} - 1$ และมีการกระจายตัวที่ดีใน 623 มิติ (623-dimension) ทำให้เหมาะสมสำหรับการใช้งานในงานคำนวณที่ต้องการตัวเลขสุ่มคุณภาพสูง เช่น งานวิทยาศาสตร์คอมพิวเตอร์ การจำลองข้อมูล (Simulation) และการเข้ารหัสข้อมูล (Cryptography)

จุดเด่นของอัลกอริทึม Mersenne Twister คือ สามารถสร้างตัวเลขสุ่มจำนวน 624 ค่าจากสถานะภายในเพียงสถานะเดียวแทนการสร้างเพียงหมายเลขเดียวต่อสถานะ ทำให้สามารถสร้างลำดับตัวเลขสุ่มได้อย่างรวดเร็วและมีประสิทธิภาพ อย่างไรก็ตาม สถานะเริ่มต้นของอัลกอริทึมจะไม่ถูกใช้ในการสร้างตัวเลขสุ่มในทันที เนื่องจากมีการปรับปรุงสถานะด้วยฟังก์ชันที่เรียกว่า Twisted function ซึ่งเป็นฟังก์ชันทางเดียว (One-way function) ซึ่งไม่สามารถย้อนกลับไปยังสถานะก่อนหน้าได้ จึงทำให้ตัวเลขสุ่มที่ได้มีความซับซ้อนและยากต่อการคาดเดา

การทำงานของอัลกอริทึม Mersenne Twister สามารถสรุปได้ดังนี้:

1. การกำหนดค่าเริ่มต้น (Initialization) : ใช้ค่าเริ่มต้น (Seed x_0) และนำไปผ่านฟังก์ชันเวียนบังเกิด (Recursive Function) เพื่อกำหนดสถานะเริ่มต้น (Initial State)
2. การใช้ Twisted function : ใช้เพื่อปรับปรุงการกระจายตัวของเลขสุ่ม ทำให้สถานะภายในมีการเปลี่ยนแปลงที่ไม่สามารถย้อนกลับได้
3. การปรับปรุงสถานะด้วย Tempering Transformation : กระบวนการนี้ทำหน้าที่ปรับความแม่นยำในการกระจายตัวของตัวเลขสุ่ม
4. การทำซ้ำกระบวนการ: กระบวนการทั้งหมดจะถูกทำซ้ำจนกว่าจะได้จำนวนตัวเลขสุ่มตามที่ต้องการ ซึ่งกระบวนการนี้สามารถเขียนออกมาเป็นฟังก์ชันเวียนบังเกิดได้ ดังนี้

$$x_{k+n} = (x_{k+m} \oplus ((x_k \& u) | (x_{k+1} \& l)))\, A \tag{2.4}$$

โดยที่ x_k คือ ลำดับของสถานะของตัวเลขสุ่มกำหนดโดยค่า Seed

n คือ จำนวนบิตของคำที่ใช้

m คือ ตำแหน่งในการเลื่อน (Shift position)

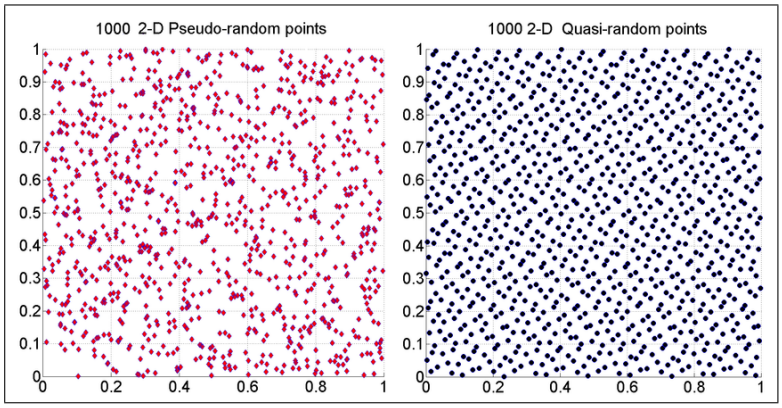
u, l คือ Bitmask ที่ใช้ในการแยกบิตสูงและต่ำ

A คือ เมตริกซ์ที่ใช้ในการแปลงค่า ซึ่งใช้เพื่อปรับเปลี่ยนลำดับตัวเลขสุ่มในขั้นตอนการทำงานของอัลกอริทึม

- Quasirandom number generator

ชุดเลขสุ่ม Quasirandom หรือ ลำดับที่มีความคลาดเคลื่อนต่ำ (Low-Discrepancy Sequences: LDS) เป็นชุดของตัวเลขที่ออกแบบมาให้มีระยะห่างระหว่างข้อมูลสม่ำเสมอกันมากขึ้น เมื่อเทียบกับเลขสุ่มเทียมรูปแบบอื่น ๆ

โดยเลขสุ่ม Quasirandom [17] ถูกพัฒนาขึ้นโดยมีเป้าหมายในการสร้างเพื่อป้องกันการกระจุกตัวของเลขสุ่มและช่องว่างระหว่างเลขสุ่มโดย แตกต่างกับเลขสุ่มเทียมปกติที่เน้นการจำลองความเป็นเลขสุ่มที่ไม่สามารถถูกคาดเดาได้ ทำให้การกระจายของเลขสุ่ม Quasirandom ทั่วถึงมากกว่าภายในปริภูมิ โดยสามารถดูได้จากรูปที่ 2.3 ที่แสดงการกระจายตัวของเลขสุ่มขนาด 2 มิติ ซึ่งภาพด้านซ้ายเป็นเลขสุ่มเทียม และภาพด้านขวาเป็นเลขสุ่ม Quasirandom



รูปที่ 2.3 การเปรียบเทียบเลขสุ่มขนาด 2 มิติระหว่างเลขสุ่มเทียม และ เลขสุ่ม Quasirandom

(ที่มา: [18])

Quasirandom ในลูกบาศก์ s มิติ หมายถึง ลำดับที่มีค่า Discrepancy อยู่ในขนาดประมาณ $(logN)^S N^{-1}$ เมื่อ N มีค่ามาก ซึ่งเป็นขนาดขั้นต่ำที่เป็นไปได้ จึงทำให้ Quasirandom มีการกระจายตัวที่สม่ำเสมอกว่าเลขสุ่มเทียม เนื่องจากเลขสุ่มเทียมมักจะเกิดการกระจุกตัว ส่งผลให้ค่า Discrepancy มีขนาดประมาณ $(loglogN)^{1/2}N^{-1/2}$ สำหรับเลขสุ่มเทียมแบบอื่น ๆ ซึ่งค่า L_∞-Discrepancy สามารถคำนวณได้จากสมการ ดังนี้

$$D_N^* = \sup_{J \in \mathcal{J}^*} \left| \frac{A(J;P)}{N} - \lambda_s(J) \right| \tag{2.5}$$

โดยที่ J คือ ช่วงสี่เหลี่ยมใน s-มิติที่มีมุมหนึ่งอยู่ที่จุดกำเนิด

A(J;P) คือ จำนวนจุดในชุด P ที่อยู่ในช่วง J

$\lambda_s(J)$ คือ ปริมาตรของช่วง J

2.2.2 เครื่องกำเนิดเลขสุ่มจริง (True random Number Generator: TRNG)

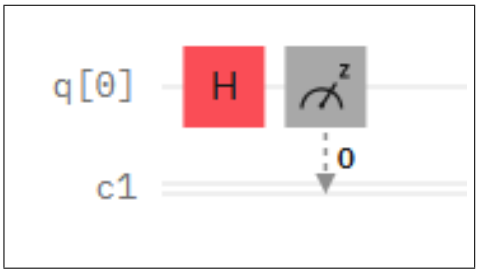
เครื่องกำเนิดเลขสุ่มจริงเป็นการสร้างเลขสุ่มจากอุปกรณ์ มีคุณสมบัติเป็นฟังก์ชันที่ขึ้นกับเวลาในปัจจุบัน และไม่สามารถคาดเดารูปแบบการเกิดขึ้นของเลขสุ่มได้ โดยในการศึกษาครั้งนี้ คณะผู้จัดทำสนใจศึกษาเครื่องกำเนิดเลขสุ่มจริงเชิงควอนตัมทั้งหมด 2 ชนิด ได้แก่

- เครื่องกำเนิดเลขสุ่มจริงเชิงควอนตัมจาก IBM ควอนตัมคอมพิวเตอร์ (IBM QRNG)

เลขสุ่มควอนตัมชนิดนี้ เป็นเลขสุ่มที่สร้างขึ้นตามหลักการกลศาสตร์ควอนตัมของวงจรควอนตัมที่ทำงานได้ในควอนตัมคอมพิวเตอร์ ซึ่งมีการกระจายตัวอย่างสม่ำเสมอ โดยเลขสุ่มควอนตัมนี้ถูกสร้างขึ้นโดยใช้วงจรที่มี Hadamard gate เป็นองค์ประกอบ [19] และทำงานอยู่บนแพลตฟอร์ม IBM Quantum Platform

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \tag{2.6}$$

การออกแบบวงจรเบื้องต้น (รูปที่ 2.4) จะใช้ Hadamard gate เพื่อสร้างสถานะ Superposition ซึ่งทำให้คิวบิต (Qubit) อยู่ในสถานะ 0 และ 1 พร้อมกัน จนกว่าจะมีการวัดค่า (Measurement) เมื่อทำการวัด ค่าที่อ่านได้จะถูกตีความเป็นเลข 0 หรือ 1 อย่างใดอย่างหนึ่งในลักษณะที่คาดเดาไม่ได้ ทำให้โอกาสที่ผลลัพธ์จะเป็น 0 และ 1 มีค่าเท่ากัน [20]

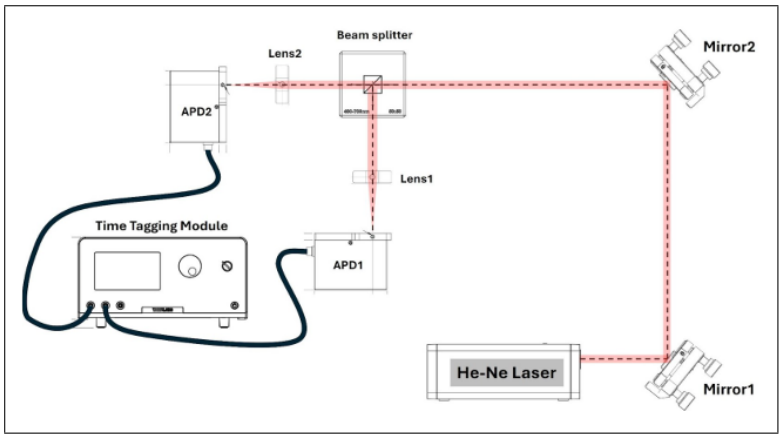


รูปที่ 2.4 วงจรเลขสุ่มเชิงควอนตัมขนาด 1 คิวบิต

- เลขสุ่มจริงจากการทดลองแสงเชิงควอนตัม (QRNG from quantum optics experiment)

เลขสุ่มจริงจากการทดลองแสงเชิงควอนตัมสามารถถูกสร้างได้จากการทดลองจำแนกลำแสงจากแหล่งกำเนิดแสงใด ๆ ที่มีความสุ่มด้วยตัวจำแนกแสง (Beam splitter) ซึ่งจะได้ลำแสงทั้งหมด 2 ลำแสง เมื่อลำแสงทั้งสองไปกระทบฉากรับ หรือเครื่องวัดอนุภาคโฟตอนที่มีความละเอียดในการระบุเวลาระดับพิโควินาที ในทางทฤษฎี การทดลองจะไม่มีโอกาสเกิดการพบอนุภาคโฟตอนกระทบฉากรับพร้อมกันทั้งสองฉากรับ (รูปที่ 2.5) เนื่องจากเป็นแสงที่มาจากแหล่งกำเนิดเดียวกัน แล้วทำให้ค่าสถานะของแสงทั้งสองที่กระทบฉาก ณ เวลาหนึ่ง ๆ เป็นสถานะซ้อนทับ มีความสุ่มโดยธรรมชาติจากแหล่งกำเนิดแสงตลอดเวลา

ในการสร้างเลขสุ่มจากการทดลอง จะสนใจผลต่างระยะเวลาที่อนุภาคโฟตอนตกกระทบระหว่างฉากรับทั้งสอง ซึ่งระยะเวลาที่ได้ทั้งหมดจะมีความละเอียดตามเครื่องมือวัดอนุภาคโฟตอน และมีการแจกแจงแบบเลขชี้กำลัง (Exponential distribution) ทำให้ต้องมีการจัดการข้อมูลเพื่อเปลี่ยนแปลงระยะเวลาให้มีการกระจายตัวในรูปแบบการแจกแจงเอกรูป แล้วแปลผลเป็นเลขสุ่ม โดยผลต่างระยะเวลาระหว่างฉากรับ 1 ค่า สามารถผลิตเลขสุ่มได้สูงสุด 16 บิต



รูปที่ 2.5 การผลิตเลขสุ่มเชิงควอนตัมจากการทดลองจำแนกลำแสง

2.3 ปัญหาการหาค่าที่เหมาะสมที่สุด

2.3.1 การทดสอบด้วยฟังก์ชันคณิตศาสตร์

ฟังก์ชันคณิตศาสตร์สามารถถูกใช้เป็นปัญหาในการหาค่าเหมาะสมที่สุดได้ โดยใช้อัลกอริทึมในการค้นหาจุดต่ำสุดหรือจุดสูงสุดของฟังก์ชันเป้าหมาย เพื่อทดสอบประสิทธิภาพของอัลกอริทึม ในการศึกษานี้ ผู้จัดทำเลือกใช้ชุดฟังก์ชันทดสอบมาตรฐานจากการแข่งขัน CEC2021 ซึ่งประกอบด้วยฟังก์ชันจำนวน 10 ฟังก์ชัน (F1–F10) ที่ออกแบบมาเพื่อสะท้อนความหลากหลายของลักษณะปัญหาที่มีพื้นผิวคำตอบ (Landscape) ที่ซับซ้อนและไม่ซับซ้อน

ฟังก์ชันในชุดนี้แบ่งออกเป็นสี่กลุ่มหลัก ได้แก่ ฟังก์ชันแบบ unimodal เช่น F1 ซึ่งมีจุดเหมาะสมที่สุดเพียงจุดเดียว, ฟังก์ชันแบบ multimodal เช่น F2–F4 ที่มีหลายจุดเหมาะสมย่อย, ฟังก์ชันแบบ hybrid เช่น F5–F7 ที่รวมลักษณะของฟังก์ชันพื้นฐานหลายรูปแบบเข้าด้วยกัน และ ฟังก์ชันแบบ composition เช่น F8–F10 ซึ่งรวมฟังก์ชันย่อยเข้าด้วยกันผ่านการปรับค่าน้ำหนักและตำแหน่งอย่างซับซ้อน

เพื่อเพิ่มความท้าทาย ชุดฟังก์ชันนี้ยังมีการเลื่อน (shift) ตำแหน่งจุดเหมาะสมที่สุด และหมุน (rotate) พิกัดของฟังก์ชัน ไม่ให้ตรงกับจุดกำเนิดหรือแกนพิกัดดั้งเดิม พร้อมทั้งอาจมีการเพิ่ม bias ต่อค่าฟังก์ชัน ทำให้พื้นผิวคำตอบ ของปัญหามีความหลอกลวงและยากต่อการค้นหา โดยทั้งหมดนี้สามารถเรียกใช้งานผ่านไลบรารี OPFUNU [21] ที่รวบรวมและจัดการฟังก์ชันปัญหา CEC2021 อย่างครบถ้วนในรูปแบบพร้อมใช้งาน

เหตุผลที่เลือกใช้ชุดปัญหา CEC2021: ชุดฟังก์ชัน CEC2021 ได้รับการออกแบบมาเพื่อใช้เป็นเกณฑ์มาตรฐานในการทดสอบประสิทธิภาพของอัลกอริทึมเชิงเมตาฮิวริสติกอย่างเป็นระบบ โดยครอบคลุมลักษณะปัญหาที่หลากหลาย

2.3.2 ปัญหาการเดินทางของพนักงานขาย (TSP)

TSP เป็นปัญหาการค้นหาเส้นทางที่สั้นที่สุดสำหรับการเดินทางผ่านเมืองทุกเมืองอย่างครบถ้วน โดยแต่ละเมืองถูกเดินทางผ่านเพียงครั้งเดียวแล้วไม่มีการกลับมายังจุดเริ่มต้น [22] ปัญหานี้จัดเป็นปัญหา NP-hard ที่มีความซับซ้อนสูง เนื่องจากจำนวนเส้นทางเป็นไปได้เติบโตอย่างรวดเร็วเมื่อจำนวนเมืองเพิ่มขึ้น ในการทดลองนี้ใช้ข้อมูลจาก TSPLIB [23] ที่มีการใช้ระยะทางแบบยูคลิดสองมิติ (EUC_2D) ในการคำนวณระยะทางระหว่างเมืองบนระนาบสองมิติ โดยถ้าเมือง i และ j มีพิกัดเป็น (x_i, y_i) และ (x_j, y_j) ตามลำดับ จะได้ระยะทางระหว่างสองเมืองใด ๆ เป็น :

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \tag{2.7}$$

เหตุผลที่เลือกใช้ TSP: ปัญหา TSP มีลักษณะปัญหาที่มนุษย์สามารถเข้าใจได้ง่ายแต่แก้ไขได้ยาก จึงมักใช้เป็นตัวแทนสำหรับทดสอบประสิทธิภาพของอัลกอริทึมเชิงประมาณ หรือ อัลกอริทึมแบบเมตาฮิวริสติกในงานวิจัยต่าง ๆ นอกจากนี้การใช้ระยะทางแบบ Euclidean ทำให้สถานการณ์สมจริงและการคำนวณระยะทางทำได้ง่ายอีกด้วย

2.3.3 ปัญหาการหาจุดตัดที่มากที่สุด (ปัญหา Max-cut)

Max-Cut เป็นปัญหาการแบ่งจุดภายในกราฟแบบไม่ถ่วงน้ำหนักและไม่ระบุทิศทาง (Unweighted undirected graph) เป้าหมายหลัก คือ แบ่งจุดในกราฟออกเป็นสองกลุ่ม เพื่อให้จำนวนเส้นเชื่อมที่จุดระหว่างสองกลุ่มมีค่ามากที่สุด เนื่องจากจำนวนวิธีแบ่งกลุ่มของจุดทั้งหมดมีแนวโน้มเพิ่มขึ้นอย่างทวีคูณเมื่อจำนวนจุดมากขึ้น ปัญหานี้จึงเป็นหนึ่งในปัญหาที่มีความซับซ้อนสูงระดับ NP-hard ในเชิงคำนวณ โดยนิยามปัญหา Max-Cut โดยใช้ adjacency matrix ของกราฟดังนี้ ให้กราฟ $G = (V, E)$ มีจำนวนจุด n และกำหนดให้ $A \in 0, 1^{n \times n}$ เป็นเมทริกซ์ติดกัน โดย $A_{ij} = 1$ ก็ต่อเมื่อมีเส้นเชื่อมระหว่างจุด i และ j และ $A_{ij} = 0$ หากไม่มีเส้นเชื่อม กำหนดตัวแปร $x_i \in 0, 1$ สำหรับจุด i แต่ละตัว เพื่อระบุว่าจุด i อยู่ในกลุ่มใด ตัวอย่างเช่น $x_i = 0$ และ 1 หมายถึงจุด i อยู่ในกลุ่มที่ 1 และ 2 ตามลำดับ เนื่องจากกราฟเป็นแบบไม่ระบุทิศทาง เราจะพิจารณาเฉพาะคู่จุด (i, j) ที่ $i < j$ ในการคำนวณค่า cut เพื่อไม่ให้นับเส้นเชื่อมเดียวกันซ้ำซ้อน สำหรับคู่จุด (i, j) ใด ๆ หาก $x_i \neq x_j$ และ $A_{ij} = 1$ (กล่าวคือมีเส้นเชื่อมระหว่าง i กับ j จริง) จะทำการนับเส้นเชื่อมนั้นเป็น 1 หน่วยในการคำนวณค่า cut ดังนั้น จำนวนเส้นเชื่อมทั้งหมดที่เชื่อมระหว่างจุดสองกลุ่ม (หรือค่า cut) สามารถคำนวณได้จากสมการ 2.8

$$\text{Cut}(x) = \sum_{i=1}^{n} \sum_{j=i+1}^{n} A_{ij} \cdot \mathbf{1}(x_i \neq x_j), \tag{2.8}$$

โดยที่ $\mathbf{1}(x_i \neq x_j)$ คือ ฟังก์ชันตัวบ่งชี้ซึ่งมีค่า 1 เมื่อ x_i ไม่เท่ากับ x_j และมีค่า 0 เมื่อ x_i เท่ากับ x_j. เป้าหมายของปัญหา Max-Cut คือการกำหนดค่า $x_1, x_2, \ldots, x_n$ (หรือการแบ่งจุดออกเป็นสองกลุ่ม) ให้ได้วิธีที่ทำให้ค่า $\text{Cut}(x)$ นี้มีค่าสูงที่สุด

เหตุผลที่เลือกใช้ Max-Cut: ปัญหา Max-Cut เป็นปัญหาเชิงกราฟที่ได้รับความนิยม โดยเฉพาะในการศึกษาด้านอัลกอริทึมเชิงควอนตัม นอกจากนี้ยังมีชุดข้อมูลกรณีทดสอบมาตรฐานจาก *DIMACS benchmark* ที่เผยแพร่สาธารณะและได้รับการยอมรับ

2.4 ชุดเครื่องมือทดสอบการเป็นเลขสุ่มจริง (Test suite)

การเป็นเลขสุ่ม คือ การทำให้ไม่สามารถคาดการณ์ได้ว่าเลขที่จะเกิดขึ้นถัดไปจะเป็นเลขใด ไม่ว่าจะมีปัจจัยใดเข้ามาเกี่ยวข้องก็ตาม เมื่อยกเว้นการพิจารณาการคำนวณจากค่า seed และการตั้งค่าของ PRNG การวัดผลคุณสมบัติการสุ่มของเลขสุ่มจากเครื่องกำเนิด

เลขสุ่มชนิดต่าง ๆ จำเป็นต้องมีการตรวจสอบในหลายปัจจัย ดังนั้น คณะผู้จัดทำจึงได้ค้นหาวิธีการตรวจสอบที่เป็นที่ยอมรับและน่าเชื่อถือในปัจจุบัน เพื่ออธิบายความสุ่มของเลขที่ได้จากการทดลอง

2.4.1 NIST STS (National Institute of Standards and Technology Statistical Test Suite)

ชุดการทดสอบ NIST STS [24] ถูกออกแบบเพื่อทดสอบคุณสมบัติความสุ่มของเลขสุ่มในระดับบิต โดยมีจุดประสงค์เพื่อใช้วิเคราะห์ความปลอดภัยการนำไปใช้เชิงวิทยาการรหัสลับ ประกอบด้วยกรณีทดสอบจำนวนทั้งหมด 15 กรณี ได้แก่

1. Frequency (Monobit) Test
   - จุดประสงค์: เพื่อตรวจสอบว่าจำนวนบิต 0 และ 1 มีจำนวนการเกิดขึ้นใกล้เคียงกันหรือไม่ (ประมาณครึ่งหนึ่งเป็น 1 และอีกครึ่งหนึ่งเป็น 0 ตามที่คาดสำหรับลำดับสุ่ม)
   - วิธีการทำ: นับจำนวนบิต 0 และ 1 ทั้งหมดในลำดับ แล้วตรวจสอบว่าจำนวนทั้งสองมีปริมาณเท่ากันหรือไม่ โดยคำนวณความเบี่ยงเบนของจำนวน 1 จากครึ่งหนึ่งของความยาวลำดับ
   - ตัวอย่าง: นับจำนวนบิต 0 และ 1 ทั้งหมดในลำดับ แล้วตรวจสอบว่าจำนวนทั้งสองเกือบเท่ากันหรือไม่ โดยคำนวณความเบี่ยงเบนของจำนวน 1 จากครึ่งหนึ่งของความยาวลำดับ (หากต่างกันมากอาจไม่สุ่ม) เช่น ถ้าลำดับ 100 บิตมี 1 จำนวน 50 ตัว และ 0 จำนวน 50 ตัว ถือว่าปกติ แต่ถ้ามี 1 ถึง 80 ตัว และ 0 แค่ 20 ตัว จะแสดงว่าไม่สมดุล (มีจำนวนการเกิดขึ้นของ 1 มากเกินไป) บ่งชี้ว่าลำดับไม่สุ่ม
2. Frequency Test within a Block
   - จุดประสงค์: เพื่อตรวจสอบว่าจำนวนบิต 1 ในแต่ละบล็อกย่อยของลำดับ (ขนาดบล็อก M บิต) มีค่าใกล้เคียงกับ $M/2$ หรือครึ่งหนึ่งของขนาดบล็อกหรือไม่
   - วิธีการทำ: แบ่งลำดับบิตออกเป็นบล็อกย่อยหลายบล็อกเท่า ๆ กัน แล้วนับจำนวน 1 ในแต่ละบล็อก จากนั้นตรวจสอบความแตกต่างของจำนวน 1 ในแต่ละบล็อกเทียบกับ $M/2$ หากบางบล็อกมี 1 มากหรือน้อยผิดปกติเมื่อเทียบกับบล็อกอื่น ๆ แสดงว่าลำดับอาจไม่สุ่ม
   - ตัวอย่าง: ถ้าแบ่งลำดับ 100 บิตออกเป็น 10 บล็อก (บล็อกละ 10 บิต) คาดว่าแต่ละบล็อกจะมี 1 ประมาณ 5 ตัว ถ้าพบว่าบล็อกหนึ่งมี 1 ถึง 9 ตัว ขณะที่บล็อกอื่นมี 1 เพียง 1 ตัว ความถี่ของ 1 ในบล็อกต่าง ๆ ไม่สม่ำเสมอ ซึ่งบ่งชี้ว่าลำดับไม่สุ่ม
3. Runs Test
   - จุดประสงค์: เพื่อตรวจสอบว่าจำนวนช่วงของบิตที่เหมือนกันเรียงติดกันในลำดับ มีจำนวนมากน้อยตามที่คาดไว้สำหรับลำดับสุ่มหรือไม่ (ไม่มากหรือน้อยเกินไป)
   - วิธีการทำ: นับจำนวน runs ต่อเนื่องของบิตที่เหมือนกัน (ทั้ง runs ของ 0 และ runs ของ 1) พร้อมทั้งพิจารณาความยาวของ runs เหล่านั้น แล้วตรวจสอบว่าโดยรวมแล้วจำนวน runs ไม่มากหรือน้อยเกินไปเมื่อเทียบกับค่าที่คาดไว้
   - ตัวอย่าง: ลำดับ 0001111000 มี runs ทั้งหมด 3 runs (0 ต่อเนื่อง 3 ตัว, 1 ต่อเนื่อง 4 ตัว, 0 ต่อเนื่อง 3 ตัว) ซึ่งจำนวน runs ค่อนข้างน้อย ในทางกลับกัน ลำดับ 0101010101 มี runs สั้น ๆ 10 runs (สลับ 0/1 ตลอด) ซึ่งมากเกินไป ทั้งสองกรณีแสดงถึงความไม่ปกติ เมื่อเทียบกันลำดับสุ่มจริงจะมีจำนวน runs อยู่ระหว่างสองกรณีนี้
4. Tests for the Longest-Run-of-Ones in a Block
   - จุดประสงค์: เพื่อตรวจสอบว่าความยาวของช่วงที่เป็น 1 ติดกันที่ยาวที่สุดภายในบล็อกขนาด M บิต เป็นไปตามที่คาดไว้สำหรับลำดับสุ่มหรือไม่ (ไม่ยาวหรือสั้นเกินไปอย่างมีนัยสำคัญ)
   - วิธีการทำ: แบ่งลำดับเป็นบล็อกละ M บิต แล้วหาว่าในแต่ละบล็อกนั้นมี runs ของ 1 ที่ยาวที่สุดกี่ตัวติดกัน จากนั้นพิจารณาการกระจายของความยาว runs ที่ยาวที่สุดเหล่านี้ว่าเข้ากับค่าที่คาดหมายหรือไม่ หากพบว่ามีบล็อกที่ runs ของ 1 ยาวมากผิดปกติ หรือทุกบล็อกมี runs ของ 1 สั้นผิดปกติ ก็บ่งชี้ความไม่สุ่ม

- ตัวอย่าง: หากบล็อกมีขนาด 20 บิต โดยทั่วไปลำดับสุ่มอาจมี runs ของ 1 ที่ยาวที่สุดในแต่ละบล็อกประมาณ 3-7 ตัวติดกัน ถ้าพบว่าบล็อกหนึ่งมี 1 ติดกันยาวถึง 15 ตัว หรือในทางกลับกัน บล็อกทุกบล็อกมี 1 ติดกันไม่เกิน 2 ตัว ก็แสดงว่าลำดับอาจไม่สุ่ม

5. Binary Matrix Rank Test

- จุดประสงค์: เพื่อตรวจสอบความเป็นอิสระเชิงเส้น (Linear dependence) ในลำดับ โดยดูอันดับ (Rank) ของเมทริกซ์ที่สร้างจากลำดับว่ามีค่าเต็มอันดับ (Full rank matrix) หรือใกล้เคียงเต็มอันดับ การมีอันดับต่ำเป็นประจำ อาจหมายถึงมีความสัมพันธ์เชิงเส้นแฝงอยู่
- วิธีการทำ: นำบิตจากลำดับมาจัดเรียงเป็นเมทริกซ์ขนาดที่กำหนด แล้วคำนวณอันดับของเมทริกซ์ (จำนวนแถวที่ไม่ขึ้นต่อกันเชิงเส้น) สำหรับลำดับสุ่ม เมทริกซ์ที่ได้มักมีอันดับสูงหรือเต็ม (แถวส่วนใหญ่เป็นอิสระต่อกัน) แต่ถ้าลำดับมีรูปแบบซ้ำหรือขึ้นต่อกัน อันดับจะต่ำกว่าที่คาด
- ตัวอย่าง: หากนำลำดับ 64 บิตมาสร้างเมทริกซ์ 8x8 แล้วพบว่าอันดับของเมทริกซ์เท่ากับ 5 (จากที่อันดับเต็มควรเป็น 8) อาจแปลว่าลำดับมีความซ้ำซ้อนเชิงเส้น (บางแถวเป็นผลผสมของแถวอื่น) ทำให้ไม่สุ่มอย่างแท้จริง

6. Discrete Fourier Transform (Spectral) Test

- จุดประสงค์: เพื่อตรวจสอบว่าลำดับมีลักษณะเป็นคาบ (Periodic features) หรือรูปแบบที่เกิดซ้ำอย่างมีนัยสำคัญหรือไม่ โดยดูจากความสูงของจุดยอด (Peaks) ในผลของ Discrete Fourier Transform
- วิธีการทำ: แปลงลำดับบิต (โดยแทน 0 เป็น -1 และ 1 เป็น +1) เป็น frequency domain ด้วยการทำ Discrete Fourier Transform จากนั้นพิจารณาค่าสเปกตรัมที่ได้ โดยดูจำนวนและขนาดของยอดคลื่นความถี่ หากมียอดสูงผิดปกติที่ความถี่ใด แสดงว่าลำดับมีรูปแบบซ้ำคาบที่ความถี่นั้นไม่สุ่ม
- ตัวอย่าง: ลำดับที่เป็น 01 สลับกันไปมาตลอด (01010101...) จะให้สเปกตรัมที่มียอดโดดเด่นที่ความถี่ของคาบ 2 เนื่องจากมีรูปแบบเกิดซ้ำทุก 2 บิต ขณะที่ลำดับสุ่มจริง สเปกตรัมจะกระจายและไม่มีจุดยอดสูงเด่นชัด

7. Non-overlapping Template Matching Test

- จุดประสงค์: เพื่อตรวจสอบว่าลำดับมีการเกิดซ้ำของรูปแบบบิตย่อย (แม่แบบ) ที่กำหนด ในปริมาณมากหรือน้อยเกินไปหรือไม่ โดยการนับจะไม่อนุญาตให้รูปแบบที่ตรวจพบทับซ้อนกัน
- วิธีการทำ: กำหนดแม่แบบบิตขนาด m (เช่น "1001") จากนั้นเลื่อนหน้าต่างขนาด m ไปตามลำดับ (โดยเมื่อพบรูปแบบจะเลื่อนข้ามไปหลังรูปแบบนั้นทันที) พร้อมนับจำนวนครั้งที่พบแม่แบบนี้ทั้งหมด ลำดับสุ่มควรพบรูปแบบตามแม่แบบในจำนวนที่ไม่มากหรือน้อยเกินไปเมื่อเทียบกับค่าคาดหมาย
- ตัวอย่าง: หากแม่แบบคือ "111" ในลำดับหนึ่งจำนวน N บิต ตามหลักความน่าจะเป็นควรพบ "111" ประมาณ $N/8$ ครั้ง (เพราะโอกาสเกิด 111 ต่อเนื่องคือ $1/8$) ถ้าพบการเกิดขึ้นซ้ำหลายครั้งของค่านี้ หรือไม่พบเลย ก็แสดงความผิดปกติ (ลำดับอาจไม่สุ่ม)

8. Overlapping Template Matching Test

- จุดประสงค์: คล้ายกับการทดสอบแม่แบบไม่ซ้อนทับ แต่ในที่นี้อนุญาตให้การค้นหารูปแบบสามารถซ้อนทับกันได้ โดยเน้นรูปแบบที่เป็นคาบ (periodic pattern) ของบิต 1 ในลำดับ เช่น รูปแบบ "1" ซ้ำ
- วิธีการทำ: ใช้วิธีเลื่อนหน้าต่าง m บิตผ่านลำดับเช่นเดียวกับ Non-overlapping Template Matching Test แต่ต่างกันที่เมื่อพบรูปแบบที่ต้องการแล้ว หน้าต่างจะเลื่อนไปเพียง 1 ตำแหน่ง (แทนการข้ามไปหลังรูปแบบ) ทำให้สามารถนับการเกิดของรูปแบบที่ซ้อนทับกันได้ ลำดับสุ่มควรมีจำนวนการเกิดรูปแบบซ้อนทับนี้ในเกณฑ์ปกติ เกิดขึ้นมากหรือน้อยกว่าที่ควรเมื่อเทียบกับค่าที่คาดไว้ จะไม่ผ่านการทดสอบนี้
- ตัวอย่าง: หากต้องการตรวจรูปแบบ "11" (1 สองตัวติดกัน) แบบ overlapping ในลำดับ "11110" การนับแบบซ้อนทับจะได้ "11" ทั้งหมด 3 ครั้ง (ตำแหน่งที่ 1-2, 2-3, 3-4) โดยค่าคาดหวังเกิดจากโอกาสที่จะเกิดแม่แบบคูณกับจำนวนการเดิน เช่นในกรณีนี้คือ $0.25 \times 5 - 2 + 1 = 1$ ลำดับที่มี "11" นั้นมีมากเกินกว่าค่าที่คาดหวังไว้อย่างมาก ลำดับอาจไม่สุ่ม

9. Maurer's "Universal Statistical" Test
    - จุดประสงค์: เพื่อตรวจสอบว่าลำดับสามารถถูกบีบอัดข้อมูลได้มากน้อยเพียงใดโดยไม่สูญเสียข้อมูล หากลำดับบีบอัดได้มาก (มีรูปแบบซ้ำที่ทำให้ย่อข้อมูลได้) ลำดับอาจไม่สุ่ม
10. Linear Complexity Test
    - จุดประสงค์: เพื่อตรวจสอบว่าลำดับมีความซับซ้อนเชิงเส้นเพียงพอที่จะถือว่าเป็นลำดับสุ่มหรือไม่ โดยพิจารณาว่าลำดับนั้นสามารถสร้างขึ้นได้ด้วย Linear Feedback Shift Register (LFSR) ที่สั้นเกินไปหรือไม่ ถ้าสามารถใช้ LFSR ที่สั้นมากได้ แสดงว่าลำดับมีรูปแบบที่ชัดเจนและไม่สุ่ม
11. Serial Test
    - จุดประสงค์: เพื่อตรวจสอบว่ารูปแบบบิตทุกแบบที่มีความยาว m (เช่น $m = 2$ จะได้ 00, 01, 10, 11) ปรากฏในลำดับด้วยความถี่ที่สม่ำเสมอหรือไม่ ซึ่งเป็นสิ่งที่ควรเกิดขึ้นในลำดับที่สุ่มจริง ๆ
12. Approximate Entropy Test
    - จุดประสงค์: เพื่อตรวจสอบระดับความไม่แน่นอน (Entropy) ของลำดับ โดยเปรียบเทียบความถี่ของรูปแบบบิตที่เกิดขึ้นในลำดับสำหรับขนาดบล็อก m และ $m + 1$ หากความต่างมีนัยสำคัญ แสดงว่าลำดับมีรูปแบบที่คาดเดาได้
13. Cumulative Sums (Cusums) Test
    - จุดประสงค์: เพื่อตรวจสอบว่าลำดับมีแนวโน้มเอนเอียงไปทางใดทางหนึ่งมากเกินไปหรือไม่ โดยดูจากค่าผลรวมสะสมของลำดับที่เปลี่ยนค่าบิตเป็น +1 และ -1 แล้วสะสมแบบต่อเนื่อง ลำดับที่สุ่มจริงควรมีผลรวมสะสมขึ้นลงใกล้ศูนย์อย่างต่อเนื่อง
14. Random Excursions Test
15. Random Excursions Variant Test

2.5 เครื่องมือสำหรับวิเคราะห์ผลการทดลอง (Experiment analytic tools)

หลังจากการทดลอง คณะผู้จัดได้ทำการบันทึกผลให้อยู่ในรูปแบบของไฟล์ข้อความ และใช้เครื่องมือการทำความสะอาด และวิเคราะห์ผลข้อมูล โดยมีรายละเอียด ดังนี้

- NumPy ใช้เพื่อแปลงชนิดข้อมูลจากไฟล์ข้อความ ให้อยู่ในรูปแบบของเมทริกซ์ที่สามารถคำนวณคณิตศาสตร์ได้มีประสิทธิภาพ
- Pandas ใช้เพื่อแปลงเมทริกซ์ที่ได้เป็น Dataframe ที่สามารถระบุแถว หรือ คอลัมน์ ได้อย่างสะดวก และใช้ตรวจสอบข้อผิดพลาดของการอ่านไฟล์ข้อความจากไลบรารี NumPy
- Seaborn และ Matplotlib ใช้เพื่อสร้างกราฟความสัมพันธ์ของตัวแปรต่าง ๆ ในการวิเคราะห์ผลลัพธ์จากการทดลอง

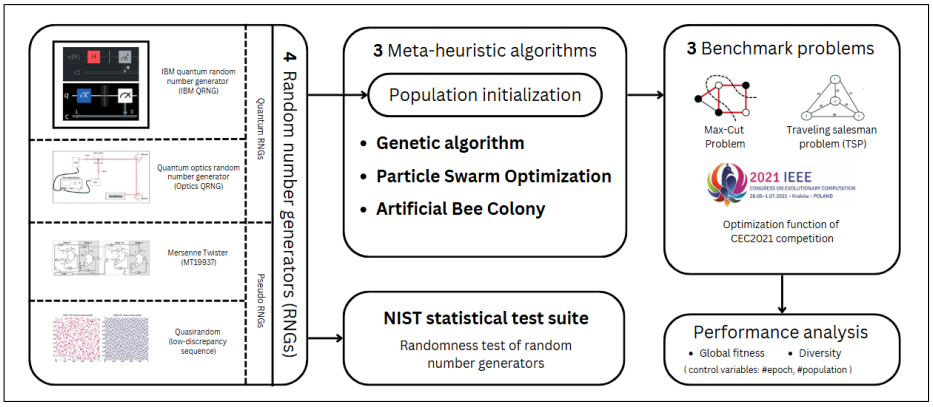
2.6 ผลกระทบของการเปลี่ยนแปลงวิธีการสร้างประชากรเริ่มต้นต่อการทำงานของอัลกอริทึมแบบเมตาฮิวริสติก

การศึกษาผลกระทบของการเปลี่ยนแปลงวิธีสร้างประชากรเริ่มต้นต่อการทำงานของอัลกอริทึมแบบเมตาฮิวริสติกครั้งแรกที่มีการบันทึกไว้ [25] ทำการศึกษาผลกระทบของเครื่องกำเนิดเลขสุ่มกับการเรียนรู้ของ GA และได้ผลลัพธ์บางกรณีที่การสร้างประชากรด้วย PRNG บางค่า seed สามารถทำให้ GA เรียนรู้ได้ดีขึ้นจากการสร้างประชากรเริ่มต้นที่ใกล้คำตอบ หลังจากนั้น การศึกษาผลกระทบในลักษณะที่คล้ายกันได้มีการใช้ฟังก์ชันการหาค่าที่ดีที่สุด (Optimization function) เพื่อวัดประสิทธิภาพของการเรียนรู้ของอัลกอริทึมแบบเมตาฮิวริสติก แต่ใช้วิธีการสร้างประชากรเริ่มต้นโดยใช้การแจกแจงรูปแบบต่าง ๆ แทน PRNG [5], [26] จึงเป็นหนึ่งในสาเหตุที่ผู้จัดทำสนใจในการศึกษาในครั้งนี้ ซึ่งจะใช้เครื่องกำเนิดเลขสุ่มมาสร้างประชากรเริ่มต้นแทนการแจกแจงด้วยสูตรคณิตศาสตร์ ประกอบไปด้วย PRNG ที่อัลกอริทึมแบบเมตาฮิวริสติกที่นิยมใช้งานในปัจจุบัน และ QRNG ที่เกิดจากปรากฏการณ์ธรรมชาติ เพราะผู้จัดทำเล็งเห็นว่า แนวคิดการพัฒนาอัลกอริทึมแบบเมตาฮิวริสติกส่วนใหญ่ได้รับแรงบันดาลจากความพยายามเลียนแบบธรรมชาติ จึงมีความต้องการเปลี่ยนวิธีการที่จะปรับกระบวนการสร้างประชากรให้คล้ายพฤติกรรมธรรมชาติมากขึ้น

บทที่ 3 วิธีการดำเนินงาน

ในบทนี้จะกล่าวถึงภาพรวมของการดำเนินงาน การพัฒนาเครื่องกำเนิดเลขสุ่ม การทดลอง และการวิเคราะห์ผลที่ได้จากการทดลอง โดยมีเนื้อหา ดังนี้

3.1 ภาพรวมของการดำเนินงาน



รูปที่ 3.1 กระบวนการทดลอง

การศึกษานี้มีจุดมุ่งหมายเพื่อศึกษาผลกระทบของการใช้งาน QRNG ต่ออัลกอริทึมแบบเมตาฮิสริสติกเพื่อสำรวจความเป็นไปได้ของการนำ QRNG มาใช้ประโยชน์ สำหรับซึ่งภาพรวมของการทดลองจะถูกแบ่งเป็น 3 ส่วน ซึ่งมีรายละเอียดและขั้นตอน ดังนี้

1. การสร้างเครื่องกำเนิดเลขสุ่ม 4 ชนิด ได้แก่ เครื่องกำเนิดเลขสุ่มจริงจากการทดลองแสงเชิงควอนตัม เครื่องกำเนิดเลขสุ่มจริงจาก IBM ควอนตัมคอมพิวเตอร์ เครื่องกำเนิดเลขสุ่มเทียมจากอัลกอริทึม Mersenne twister และเลขสุ่มที่ได้จากอัลกอริทึม Quasirandom และทำการตรวจคุณสมบัติความสุ่มของเลขสุ่มที่ได้จากเครื่องกำเนิดเลขสุ่มแต่ละชนิดด้วยเครื่องมือทดสอบ NIST STS

2. เลขสุ่มที่สร้างได้จะถูกใช้งานเข้ากับกระบวนการสร้างประชากรเริ่มต้นของอัลกอริทึมแบบเมตาฮิวริสติกที่ถูกพัฒนาขึ้นภายในไลบรารี MEALPY โดยมีการเปลี่ยนแปลงคำสั่งเพื่อให้สอดคล้องกับการทดลอง

3. การทดลองศึกษาผลกระทบของการสร้างประชากรเริ่มต้นด้วยเครื่องกำเนิดเลขสุ่มชนิดต่าง ๆ ต่อประสิทธิภาพการทำงานของอัลกอริทึมแบบเมตาฮิวริสติก คณะผู้จัดทำจะใช้ชุดเลขสุ่มจากเครื่องกำเนิดเลขสุ่มที่แตกต่างกันทั้งหมด 4 ชนิด ทำการศึกษาอัลกอริทึมแบบเมตาฮิวริสติก 3 ชนิดที่แตกต่างกัน ได้แก่ GA, PSO และ ABC ซึ่งนำไปใช้แก้ไขปัญหาการหาค่าเหมาะสมที่สุด 3 ชนิด ซึ่งมีรายละเอียดของกรณีที่ใช้ทดสอบ ดังนี้

    - Traveling salesman problem 5 กรณีทดสอบ ได้แก่ Bier127, Eil101, Pr76, St70 และ Berlin52
    - ปัญหาการหาค่าเหมาะสมที่สุดจากการแข่งขัน CEC2021 10 กรณีทดสอบ
    - ปัญหา Max-cut 5 กรณีทดสอบ ได้แก่ Brock200_2, Brock200_4, C125.9, gen200_p0.9_44.b และ Keller4

    ในแต่ละกรณีทดสอบจะกำหนดขนาดประชากรเริ่มต้น 100 คำตอบ และจำนวนรอบสูงสุดที่อัลกอริทึมแบบเมตาฮิวริสติกทำงาน 1000 รอบ (epoch) ซึ่งทุกกรณีทดสอบ 1 กรณีถูกทดลองซ้ำทั้งหมด 30 ครั้ง เพื่อบันทึกค่าที่สนใจในแต่ละรอบ ได้แก่

    - ค่าเหมาะสมโดยรวม (global fitness) คือ ค่าเหมาะสมของคำตอบที่แก้ไขปัญหาได้ดีที่สุดตั้งแต่เริ่มต้นจนถึงรอบใด ๆ
    - ค่าความหลากหลาย (diversity) คือ ค่าเฉลี่ยที่แสดงถึงความแตกต่างของประชากรแต่ละรอบ โดยคิดจากการหาค่าสัมบูรณ์ของผลต่างระหว่างคำตอบแต่ละคำตอบ และค่ามัธยฐานของคำตอบในแต่ละรอบการทำงาน ถ้าค่าความหลากหลายมาก จะสามารถถูกตีความว่า ประชากรในรอบการทำงานนั้นมีการกระจายตัวของคำตอบสูง

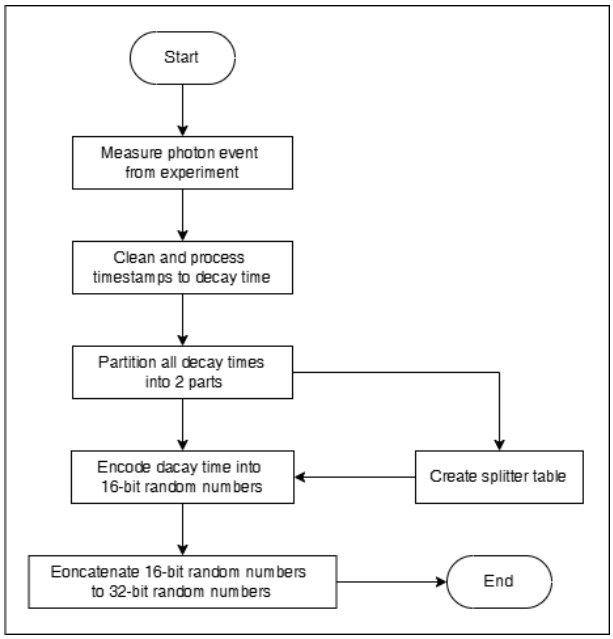
- ค่าการสำรวจ (exploration) คือ ค่าความหลายหลายที่ถูกนำมาคิดเป็นร้อยละ โดยเปรียบเทียบจากค่าความหลากหลายปัจจุบัน และค่าความหลากหลายสูงที่สุด
- เวลาที่อัลกอริทึมแบบเมตาฮิวริสติกทำงาน (runtime)
- จำนวนรอบที่อัลกอริทึมแบบเมตาฮิวริสติกทำงาน ณ ขณะนั้น

3.2 การพัฒนาเครื่องกำเนิดเลขสุ่ม

เนื่องจากการทดลองในงานวิจัยครั้งนี้ มีการใช้งานเครื่องกำเนิดเลขสุ่มจริงเชิงควอนตัมจากการทดลองเชิงกายภาพ แล้วนำผลการทดลอง มาคำนวณแปลผลเป็นเลขสุ่ม หรือเครื่องสุ่มที่คำนวณมาจากลำดับโดยเริ่มจากค่าเริ่มต้น เครื่องกำเนิดเลขสุ่มทั้งหมดไม่ได้เป็นการนำเครื่องกำเนิดเลขสุ่มจากอินเทอร์เน็ตมาใช้งาน ตัวอย่างเช่น เครื่องกำเนิดเลขสุ่มจริงจากสิ่งรบกวนบนชั้นบรรยากาศ หรือ เครื่องกำเนิดเลขสุ่มจาก API ANU QRNG [27] ดังนั้น เครื่องกำเนิดเลขสุ่มแต่ละรูปแบบจะมีขั้นตอน และรายละเอียดการสร้างแตกต่างกันที่ต้องระบุให้ชัดเจน เพื่อกำหนดให้คุณสมบัติของเลขสุ่มที่สามารถเปรียบเทียบกันได้ โดยจะเปรียบเทียบเลขสุ่มด้วยเครื่องมือ ทดสอบความสุ่มของเลขสุ่มระดับบิตด้วยเครื่องมือ NIST STS ซึ่งจะใช้ข้อมูลเลขสุ่ม 32 บิต มาเรียงต่อกันให้กลายเป็นกระแสบิต (Bit stream)

1. เครื่องกำเนิดเลขสุ่มจริงจากการทดลองแสงเชิงควอนตัม

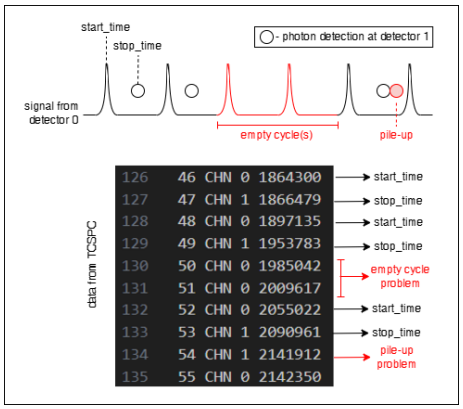
การสร้างเครื่องกำเนิดเลขสุ่มชนิดนี้ จะอาศัยหลักการการสุ่มเชิงควอนตัมของอนุภาคโฟตอนที่วัดค่าได้จากการทดลอง Single-photon Measurement โดยใช้แสงเลเซอร์เป็นแหล่งกำเนิดสัญญาณ โดยการทดลองจะสนใจผลต่างของระยะเวลาที่โฟตอนทั้งสองตัวเคลื่อนที่ไปถึงฉากรับไม่พร้อมกัน [28] ข้อมูลความต่างระยะเวลาดังกล่าวจะกระจายตัวเป็นรูปแบบการแจกแจงแบบเลขชี้กำลัง (Exponential distribution) ทำให้สามารถแปลผลกลายเป็นการแจกแจงเอกรูปได้โดยใช้สูตรการหาควอนไทล์ (Quantile) ที่ระบุค่าความน่าจะเป็นให้มีความละเอียดตามช่วงเลขสุ่มที่จะสร้างได้ โดยขั้นตอนการสร้างเลขสุ่มสามารถดูได้จากรูปที่ 3.2



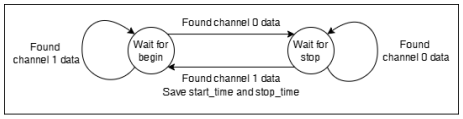
รูปที่ 3.2 ขั้นตอนของการเครื่องกำเนิดเลขสุ่มจริงจากการทดลองแสงเชิงควอนตัม

ขั้นตอนการสร้างเครื่องกำเนิดเลขสุ่มจริงจากการทดลองแสงเชิงควอนตัมมีขั้นตอน ดังนี้

(a) นำค่าเวลาที่ถูกระบุด้วย timestamp ที่ได้จาก TCSPC (Time-Correlated Single Photon Counting) มาทำความสะอาด (รูปที่ 3.4) จากกรณีปัญหาที่เกิดขึ้นหลังจากการปล่อยแสงเลเซอร์ 1 ครั้ง ได้แก่ ปัญหาการตรวจพบโฟตอนมากกว่า 1 ครั้งบนฉากรับ (Pile-up) และปัญหาที่ฉากรับไม่พบโฟตอน ข้อมูลหลังจากการทำความสะอาดจะได้ข้อมูล Timestamp ของเวลาขณะที่โฟตอนที่เคลื่อนที่เป็นเส้นตรงกระทบฉาก (start_time) และ timestamp ของเวลาขณะที่โฟตอนที่ถูกสะท้อนจาก ตัวจำแนกแสงกระทบฉากรับ (stop_time) สลับกัน (รูปที่ 3.3)

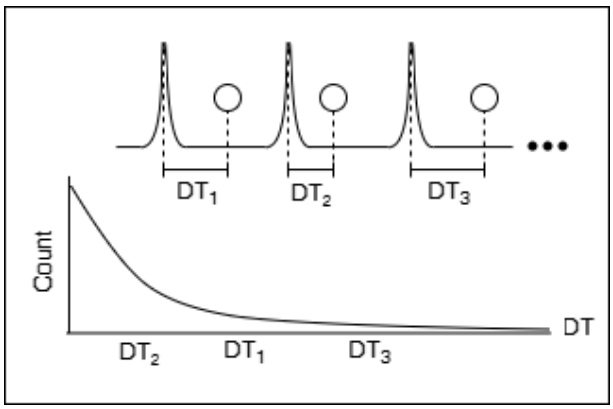


รูปที่ 3.3 ข้อมูลจากการทดลอง และปัญหาความสกปรกของข้อมูล



รูปที่ 3.4 state machine diagram ของการทำความสะอาดข้อมูล

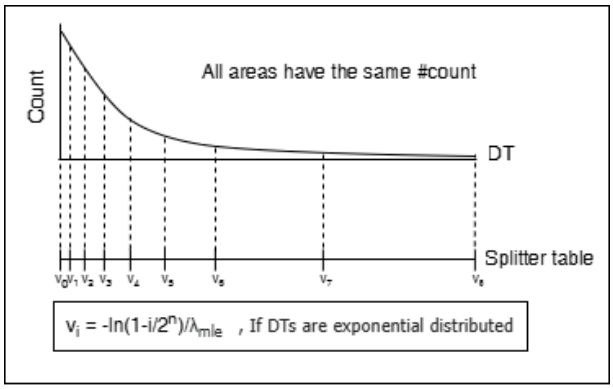
(b) นำข้อมูล start_time และ stop_time แต่ละคู่มาหาผลต่าง (DT, decay time) และจะมีการแจกแจงใกล้เคียงรูปแบบเลขชี้กำลัง (รูปที่ 3.5)



รูปที่ 3.5 ลักษณะของข้อมูลที่ถูกการทำความสะอาด

(c) ข้อมูล decay time ทั้งหมดจะถูกแบ่งเป็นสองกลุ่มเพื่อใช้สำหรับการสร้างตารางจำแนก (Splitter table) และสำหรับการสร้างเลขสุ่ม (รูปที่ 3.6)

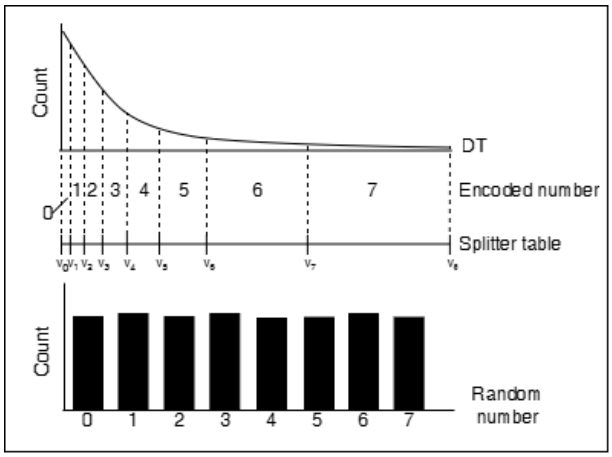
- สร้างตารางจำแนกจะทำหน้าที่คล้ายตัวเข้ารหัส (Encoder) ที่จะเปลี่ยนการกระจายตัวของข้อมูล decay time ให้กลายเป็นการแจกเจงเอกรูป โดยจะใช้ข้อมูล ได้แก่
  - ข้อมูลพิสัยของเลขสุ่มที่ต้องการสร้างต่อ 1 ค่า decay time ซึ่งสามารถกำหนดได้สูงสุด 16 บิต ตามข้อจำกัดของ decay time สูงสุด และความละเอียดสูงสุดของ timestamp ที่วัดได้โดย TCSPC (4 พิโควินาที) หากกำหนดช่วงพิสัยของเลขสุ่มขนาด n บิต ตารางจำแนกจะมีขนาด 2^n ช่อง หรือ มี 2^{n+1} ค่าที่ใช้เป็นขอบเขตบนและขอบเขตล่างในการแปลผล
  - ข้อมูล decay time ส่วนหนึ่ง ซึ่งควรมีจำนวนข้อมูลมากกว่า 1 ล้านค่า เพื่อทำให้ตารางจำแนกมีการแปลผลที่ใกล้เคียงกับข้อมูล decay time ที่ใช้สร้างเลขสุ่ม



รูปที่ 3.6 การสร้างตารางจำแนกสำหรับเลขสุ่มขนาด 3 บิต

ในอุดมคติ หากข้อมูล decay time มีการกระจายตัวแบบเลขชี้กำลัง ค่าภายในตารางจำแนกสามารถสร้างโดยใช้ควอนไทล์ได้โดยตรง แต่เนื่องจากผลการทดลองมีสิ่งรบกวนจากการทดลอง และความคลาดเคลื่อนจาก TCSPC เช่น ปัญหาเวลาที่อุปกรณ์ใช้เวลาประมวลผลข้อมูล (Dead time) คณะผู้จัดทำจึงใช้ข้อมูล decay time เป็นค่าในตารางจำแนกแทนค่าที่ได้จากการคำนวณควอนไทล์

- สร้างเลขสุ่มโดยใช้ข้อมูลที่เหลือจากการสร้างตารางจำแนก นำมาเปรียบเทียบลำดับกับค่าในตารางจำแนกโดยใช้ อัลกอริทึม Binary search (รูปที่ 3.7)



รูปที่ 3.7 การใช้ตารางจำแนกเพื่อสร้างเลขสุ่มขนาด 3 บิตต่อข้อมูล Dacay time 1 ค่า

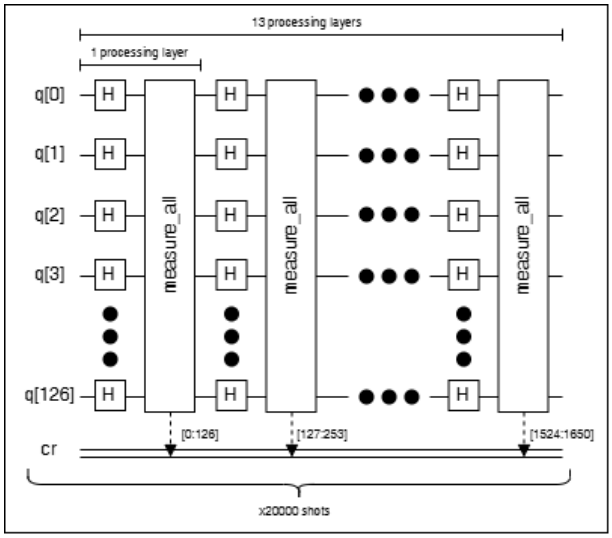
(d) นำเลขสุ่มที่สร้างได้ขนาด 16 บิต เชื่อมต่อเข้าด้วยกัน (Concatenate) ให้กลายเป็นเลขสุ่มขนาด 32 บิต

2. เครื่องกำเนิดเลขสุ่มจริงจาก IBM ควอนตัมคอมพิวเตอร์

เครื่องกำเนิดเลขสุ่มจริงจะสร้างจากวงจรควอนตัม [28] การสร้างเลขสุ่มอาศัยหลักการการสุ่มจากการวัดค่าสถานะ Superposition ของ 1 คิวบิตที่มีสถานะ 0 และ สถานะ 1 ด้วยความน่าจะเป็น 50 % เท่ากัน และเมื่อวัดค่าจะทำให้ได้ค่า 0 และ 1 สุ่มตามทฤษฎีกลศาสตร์ควอนตัม (Quantum mechanics) โดยคิวบิตทั้งหมดจะทำงานอยู่ในวงจรควอนตัมที่ทำงานในควอนตัมคอมพิวเตอร์ IBM_Brisbane ของบริษัท IBM และมีคุณสมบัติของวงจรควอนตัม (รูปที่ 3.8) ดังนี้

- วงจรควอนตัมมีขนาด 127 คิวบิต เป็นจำนวนคิวบิตสูงสุดของควอนตัมคอมพิวเตอร์ IBM_Brisbane แต่ละคิวบิตจะถูกดำเนินการด้วย Hadamard gate เพื่อเปลี่ยนสถานะ 0 ให้กลายเป็นสถานะ Superposition
- หลังจากนั้น แต่ละคิวบิตจะถูกวัดค่าไปยังหน่วยความจำแบบคลาสสิก (Classical computer) ที่มีลำดับเดียวกันเพื่อไม่ให้ค่าจะการวัดเกิดการบันทึกซ้ำในการรายงานผล

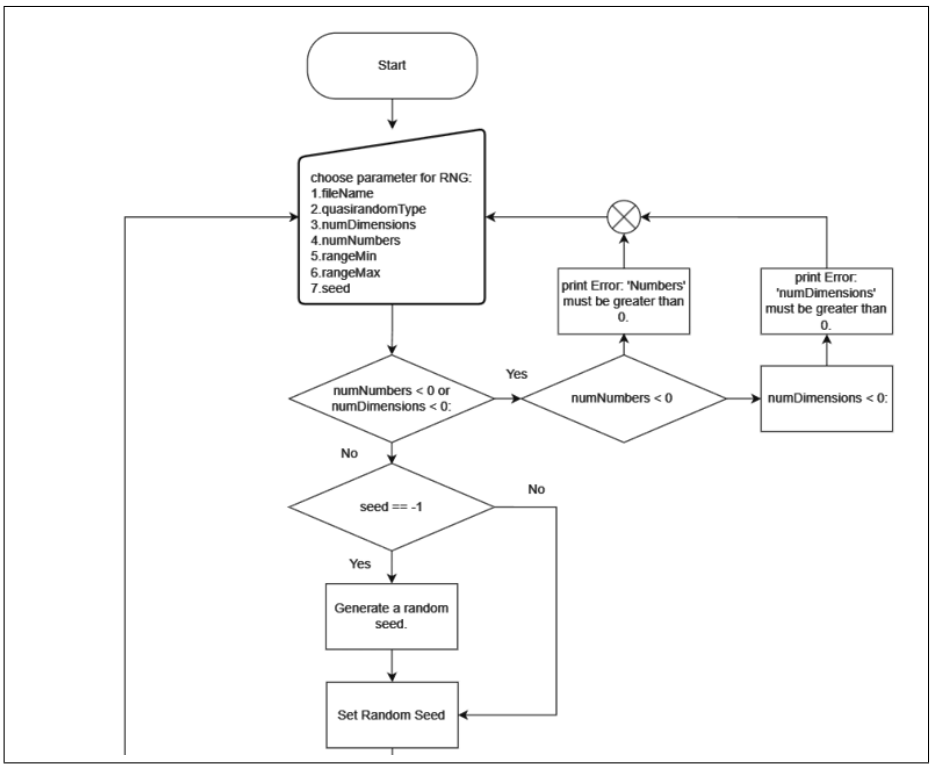
- เมื่อพิจารณาชุดของ Hadamard gate 127 คิวบิต และการวัดค่า 127 คิวบิต เป็น 1 ชั้นการทำงาน (Processing layer) สถานะของคิวบิตหลังจากการวัดจะกลายเป็น 0 หรือ 1 เท่านั้น ทำให้เราสามารถเพิ่มชั้นการทำงานเพื่อเปลี่ยนสถานะของคิวบิตให้เป็นสถานะ Superposition และวัดค่าได้อีกครั้ง ซึ่งจำนวนชั้นการทำงานของวงควอนตัมจะใช้ทั้งหมด 13 ชั้นการทำงาน ซึ่งจะใช้หน่วยความจำแบบคลาสสิกทั้งหมด 1651 ตัว
- การทำงานของควอนตัมคอมพิวเตอร์ในอัลกอริทึมต่าง ๆ จะใช้การทดลองวงจรควอนตัมตามจำนวนครั้ง (Shot) เพื่อให้ได้ผลลัพธ์เป็นค่าความน่าจะเป็นของคำตอบที่เป็นไปได้ จำนวนครั้งที่วงจรควอนตัมใช้เพื่อสร้างเลขสุ่มจะกำหนดไว้สูงสุดที่ 20000 ครั้ง เพื่อให้ได้บิตจากหน่วยความจำแบบคลาสสิกมากที่สุด โดยไม่เกินความจุหน่วยความจำต่อ 1 งาน (Job)
- นำบิตสุ่มที่อ่านได้จากหน่วยความจำแบบคลาสสิกทั้งหมดมาตัดแบ่ง (Split) ช่วงละ 32 บิต เพื่อให้ได้เลขสุ่มขนาด 32 บิต



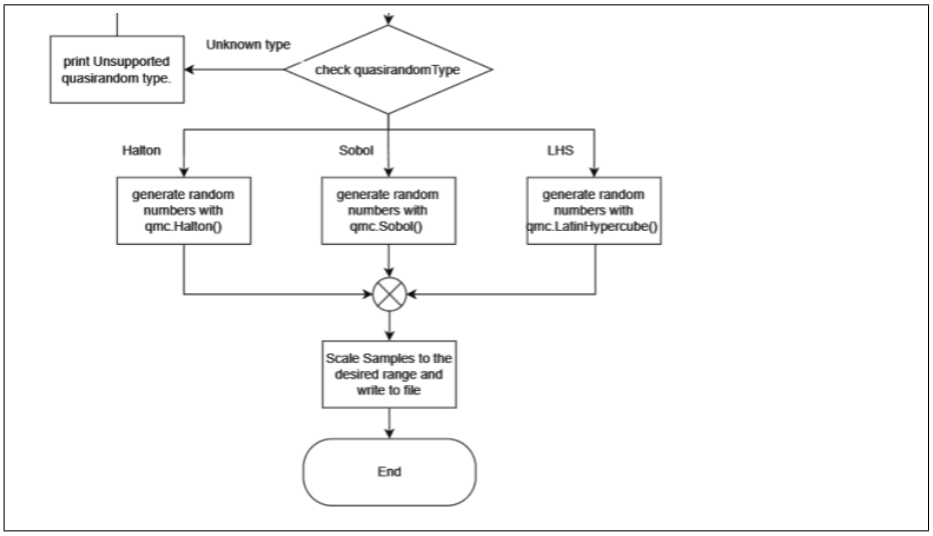
รูปที่ 3.8 วงจรควอนตัมคอมพิวเตอร์สำหรับการสร้างเลขสุ่ม

3. เครื่องกำเนิดเลขสุ่มเทียม Quasirandom

เครื่องกำเนิดเลขสุ่มเทียม Quasirandom เป็นวิธีการสร้างลำดับตัวเลขที่ดูคล้ายสุ่ม แต่ในความจริงมีโครงสร้างที่ออกแบบมาเพื่อลดความเหลื่อมล้ำ (discrepancy) ของจุดที่สุ่มในช่วงพื้นที่ตัวเลขเหมาะสำหรับงานที่ต้องการการกระจายตัวของตัวเลขอย่างสม่ำเสมอ เช่น การจำลอง Monte Carlo เป็นต้น โดยการทำงานของเครื่องกำเนิดเลขสุ่มเทียมในงานวิจัยนี้มาจากไลบรารี SciPy โดยการทำงานมีรายละเอียด ดังรูปที่ 3.9 และรูปที่ 3.10



รูปที่ 3.9 การสร้างเลขสุ่มของอัลกอริทึม Quasirandom ที่ใช้ในการทดลอง



รูปที่ 3.10 การสร้างเลขสุ่มของอัลกอริทึม Quasirandom ที่ใช้ในการทดลอง (ต่อ)

เลขสุ่มที่สร้างเพื่อไปทดสอบนั้นเป็นเลข 1 มิติ จึงใช้การสร้างเลขสุ่มเทียม Quasirandom ในรูปแบบของ Halton โดยมีหลักการสร้างเลขสุ่ม [29] ดังนี้

- เลือกฐานเฉพาะสำหรับแต่ละมิติ: สำหรับมิติที่ j ให้เลือกฐานเป็นจำนวนเฉพาะ b_j เช่น ฐาน 2 สำหรับมิติแรก ฐาน 3 สำหรับมิติที่สอง
- แปลงเลขฐานสิบเป็นฐาน b_j: สำหรับจำนวนเต็มบวก n แปลง n เป็นเลขฐาน b_j เพื่อสร้างลำดับเศษส่วนที่ใช้คำนวณ
- คำนวณค่า Radical Inverse Function: แปลงตัวเลขในฐาน b_j โดยใช้ฟังก์ชันผกผันเศษส่วน:

$$\phi_{b_j}(n) = \sum_{i=0}^{k} \frac{a_i}{b_j^{i+1}} \tag{3.1}$$

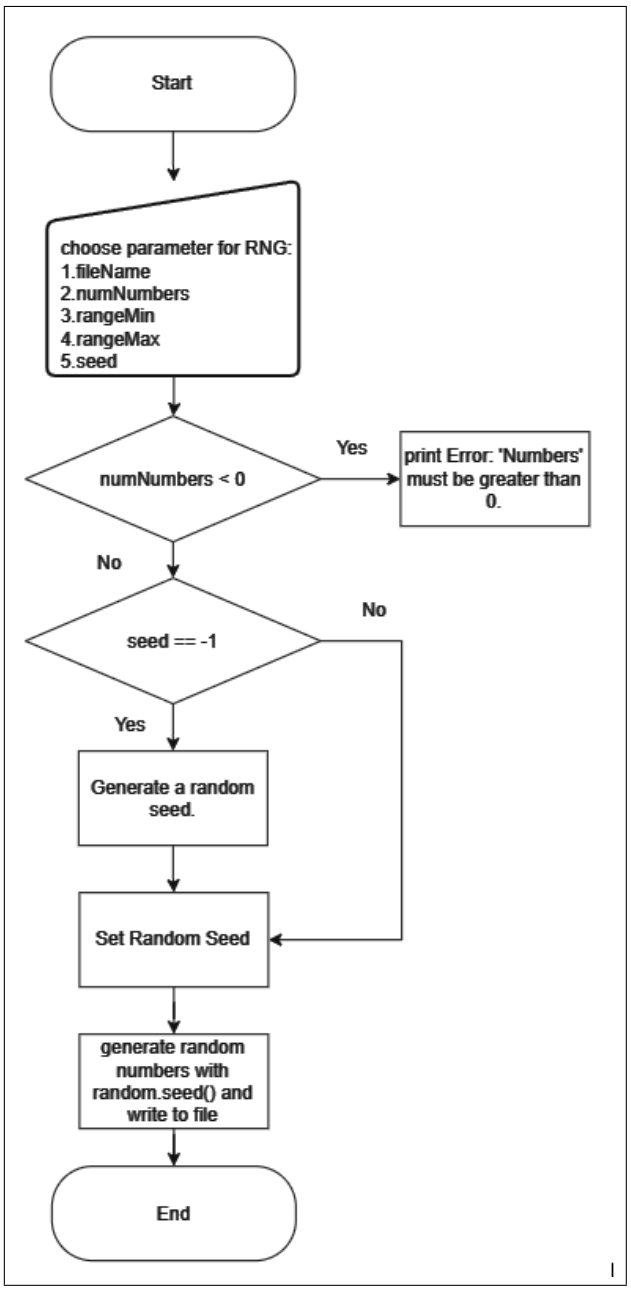
- สร้างจุดในมิติ s: จุดที่ n ใน Halton Sequence มิติ s คือ

$$\mathbf{H}_n = (\phi_{b_1}(n), \phi_{b_2}(n), \ldots, \phi_{b_s}(n)) \tag{3.2}$$

คำตอบที่ได้จะอยู่ในช่วงของ 0 ถึง 1 ทำให้ต้องมีการปรับช่วง และต้องมีการทำให้เป็นจำนวนเต็มโดยใช้เทคนิคการประมาณค่าที่ทศนิยม 1 ตำแหน่ง

4. เครื่องกำเนิดเลขสุ่มเทียม MT19937

MT19937 (Mersenne Twister) เป็นอัลกอริทึมที่ได้รับความนิยมอย่างมากในการสร้างเลขสุ่มเทียม เนื่องจากมีประสิทธิภาพสูงและช่วงการทำซ้ำ (period) ที่ยาวนานถึง $2^{19937} - 1$ จุดเด่นของอัลกอริทึมนี้คือความสามารถในการสร้างลำดับเลขสุ่มที่มีคุณสมบัติใกล้เคียงกับการสุ่มจริงในเชิงสถิติ และมีขั้นตอนการสร้างเลขสุ่ม ดังรูปที่ 3.11



รูปที่ 3.11 การสร้างเลขสุ่มเทียมจากอัลกอริทึม MT19937 ที่ใช้ในการทดลอง

3.3 การทดลอง

เนื่องจากการทดลองมีขอบเขตการศึกษากับปัญหา 3 ประเภท ได้แก่ CEC2021, TSP และ Max-cut จึงต้องระบุรายละเอียดการทดลองที่แตกต่างกันตามรูปแบบของคำตอบในแต่ละปัญหา ซึ่งประกอบไปด้วย 2 ส่วน ได้แก่ การสร้างประชากรเริ่มต้นจากเลขสุ่ม และการวัดคุณภาพประชากร

3.3.1 การสร้างประชากรเริ่มต้นจากเลขสุ่มสำหรับปัญหาแต่ละประเภท

ก่อนการดำเนินการสร้างชุดคำตอบ ได้มีการแปลงชุดตัวเลขสุ่มทั้งหมดให้อยู่ในรูปของจำนวนจริงบวกที่มีค่าระหว่าง 0 และ 1 แล้วจึงนำมาสร้างประชากรเริ่มต้น คณะผู้จัดทำได้ตัดสินใจจัดสร้างชุดประชากรที่แตกต่างกันตามลักษณะของปัญหาเพื่อหลีกเลี่ยงความลำเอียงหรืออคติที่อาจเกิดขึ้นจากชุดคำตอบที่ใช้ในการทดลอง

ในการสร้างประชากรแต่ละครั้ง จะมีการเลือกช่วงของชุดตัวเลขสุ่มที่แตกต่างกันเป็นจุดเริ่มต้น และมีการจัดเตรียมในรูปแบบที่เหมาะสมกับปัญหาแต่ละประเภท โดยลักษณะของการจัดการชุดคำตอบ แบ่งออกเป็น 3 รูปแบบ ตามชนิดของปัญหาที่นำไปใช้ ดังนี้

- **ปัญหา CEC2021**

  ประชากร 1 คำตอบ จะถูกกำหนดให้อยู่ในรูปแบบของเวกเตอร์คำตอบ (Solution vector) ที่มีขนาดเท่ากับมิติของปัญหา (Dimension) โดยองค์ประกอบเวกเตอร์มีค่าในช่วงจำนวนตั้งแต่ -100 ถึง 100 ซึ่งผู้จัดทำได้กำหนดขนาดประชากรเริ่มต้น 100 คำตอบ โดยแต่ละคำตอบเป็นเวกเตอร์ที่มีมิติเท่ากับ 10 จึงต้องใช้เลขสุ่มทั้งหมด 1000 ตัวต่อการทดลอง 1 ครั้ง โดยเลขสุ่มที่อยู่ในช่วง 0 ถึง 1 จะถูกดำเนินการโดยปรับขนาดข้อมูลแบบต่ำสุด-สูงสุด (Min-max normalization) เพื่อกระจายข้อมูลให้อยู่ภายในช่วง -100 ถึง 100 ก่อนที่จะนำเข้าสู่การทำงานของอัลกอริทึมต่อไป

- **ปัญหา TSP**

  ประชากร 1 คำตอบ จะถูกกำหนดอยู่ในรูปแบบการเรียงสับเปลี่ยนของเมืองที่ถูกแทนที่ด้วยตัวเลขจำนวนเต็มบนระนาบสองมิติ (Node Coordinates Section) ซึ่งแสดงถึงลำดับของการเดินทางระหว่างเมืองทั้งหมดโดยไม่ซ้ำกัน และใช้การคำนวณระยะทางแบบยุคลิดในสองมิติ (2D Euclidean Distance) เพื่อวัดคุณภาพของประชากร โดยจำนวนขนาดของประชากรแต่ละคำตอบจะมีค่าเท่ากับจำนวนเมืองของกรณีทดสอบ ในกรณีของปัญหา Berlin52 ซึ่งประกอบด้วย 52 เมือง สามารถใช้ตัวเลขสุ่มที่แจกแจงอย่างสม่ำเสมอในช่วง [0, 1] จำนวน 52 ค่า เพื่อสร้างลำดับเส้นทางเริ่มต้นได้ โดยให้แต่ละค่าแทนตำแหน่งของเมืองหนึ่งเมือง จากนั้นทำการ เรียงลำดับ (rank) ค่าตัวเลขสุ่มเหล่านี้ แล้วจัดเมืองตามลำดับของค่าที่ได้ เพื่อให้ได้การจัดเรียงแบบสุ่มของหมายเลขเมืองตั้งแต่ 1 ถึง 52 โดยไม่มีการซ้ำ เป็นต้น

- **ปัญหา Max-cut**

  ประชากร 1 คำตอบ จะถูกกำหนดอยู่ในรูปแบบของลำดับเลขฐานสองขนาดตามจำนวนจุดบนกราฟกรณีทดสอบ ซึ่งแต่ละบิตของเลขฐานสองที่เรียงต่อกันแทนฝั่งของจุดนั้น ๆ หลังการลากเส้นแบ่งกราฟ ตัวอย่างเช่น brock200_2 เป็นกรณีทดสอบที่ประกอบด้วย 200 จุด เนื่องจากกำหนดประชากรเริ่มต้น 100 คำตอบ จะต้องใช้บิตสุ่มทั้งหมด 20000 บิต ซึ่งมาจากการปัดค่าเลขสุ่มที่อยู่ในช่วง 0 ถึง 1 ให้กลายเป็นจำนวนเต็มจำนวน 20000 ตัว เป็นต้น

3.3.2 การวัดคุณภาพประชากร

หลังจากการสร้างประชากรเริ่มต้น ประชากรทั้งหมดจะถูกนำไปประเมินคุณภาพด้วยอัลกอริทึมที่กำหนดไว้ โดยแต่ละปัญหาจะมีฟังก์ชันประเมินค่าเหมาะสม (Fitness) ที่แตกต่างกันตามลักษณะของปัญหา ดังนี้

- **ปัญหา CEC2021**

  ชุดคำตอบจะถูกนำไปแทนค่าในฟังก์ชันของปัญหานั้นโดยตรงด้วยไลบรารี OPFUNU เพื่อคำนวณค่าผลลัพธ์ของฟังก์ชัน ซึ่งค่าดังกล่าวจะถูกใช้เป็นค่าความเหมาะสมของแต่ละคำตอบ

- **ปัญหา TSP**

  ชุดคำตอบจะระบุลำดับการเดินทางระหว่างเมือง โดยถือว่าตำแหน่งแรกในลำดับเป็นจุดเริ่มต้น และเดินทางตามลำดับเมืองที่ปรากฏในชุดคำตอบไปจนถึงเมืองสุดท้าย แล้ววนกลับมายังจุดเริ่มต้นอีกครั้ง เพื่อให้เป็นการเดินทางแบบรอบ (roundtrip) จากนั้นจะคำนวณระยะทางรวมของเส้นทางทั้งหมดโดยอ้างอิงจากพิกัดของเมืองในรูปแบบของระยะทางแบบยุคลิดสองมิติ (2D Euclidean Distance)

- **ปัญหา Max-cut**

  ในการประเมินค่าความเหมาะสมของคำตอบสำหรับปัญหานี้ จะทำการแปลงข้อมูลของกราฟให้อยู่ในรูปแบบของ adjacency matrix ซึ่งระบุความเชื่อมโยงระหว่างจุดภายในกราฟ โดยค่าที่ตำแหน่ง (i, j) ของเมทริกซ์ประชิดจะแสดงว่ามีการเชื่อมโยงระหว่างจุดที่ i และ j หรือไม่ จากนั้นจะพิจารณาคู่ของจุด i และ j ที่อยู่ในกลุ่มต่างกัน (เช่น จุด i อยู่ในกลุ่มที่แทนด้วยค่า 0 และจุด j อยู่ในกลุ่มที่แทนด้วยค่า 1) และหากตำแหน่ง (i, j) ในเมทริกซ์ประชิดมีค่าเท่ากับ 1 แสดงว่าจุดทั้งสองมีเส้นเชื่อมระหว่างกัน จะถือว่าเป็นการ cut เส้นเชื่อมนั้น และทำการบวกค่าเหมาะสม เพิ่มขึ้นอีก 1 หน่วย

ผู้จัดทำได้นำปัญหาต่าง ๆ ไปใช้ในฟังก์ชันประเมินค่าเหมาะสมที่เหมาะสมกับลักษณะของแต่ละปัญหา และดำเนินการทดสอบโดยใช้อัลกอริทึมเชิงเมตาฮิวริสติกจำนวน 3 อัลกอริทึม โดยใช้งานผ่านไลบรารี MEALPY ซึ่งเป็นไลบรารีที่ช่วยให้สามารถเรียกใช้อัลกอริทึมต่าง ๆ ได้โดยตรง และสามารถเก็บข้อมูลระหว่างการทำงานแต่ละรอบ โดยมีข้อมูลที่ถูกบันทึกไว้เพื่อศึกษา ซึ่งประกอบด้วย ค่าความหลากหลาย ค่าการสำรวจ ค่าการใช้ประโยชน์ ค่าเหมาะสมในแต่ละรอบ ค่าเหมาะสมโดยรวม และ เวลาในการประมวลผลในแต่ละรอบ

3.4 การวิเคราะห์ผลที่ได้จากการทดลอง

3.4.1 การวิเคราะห์การชนะของค่าเหมาะสมโดยรวม และความหลากหลายของประชากร

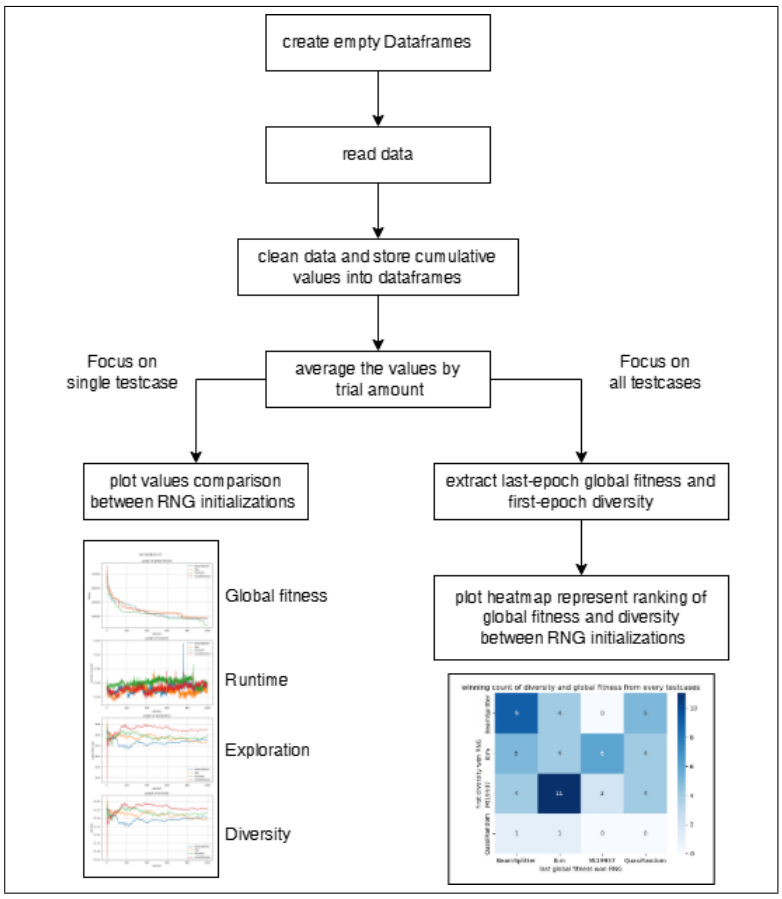
ในการสังเกตผลการทดลอง คณะผู้จัดทำได้เริ่มสังเกตจากแผนภาพแสดงจำนวนการชนะของเครื่องกำเนิดเลขสุ่มแต่ละประเภท เพื่อพิจารณาว่า QRNG หรือ PRNG สามารถสร้างประชากรที่มีข้อดีในด้านความหลากหลายของประชากรเริ่มต้น หรือสร้างประชากรเริ่มต้นที่ส่งผลดีกับประชากรคำตอบสุดท้าย หรือไม่ โดยจะแสดงเป็นแผนภาพความร้อนที่แกนระนาบเป็นประเภทของเครื่องกำเนิดเลขสุ่มที่สามารถสร้างประชากรเริ่มต้นแล้วได้ค่าเหมาะสมโดยรวมดีที่สุดในรอบสุดท้ายของการทำงาน เปรียบเทียบกับประชากรจากเครื่องกำเนิดเลขสุ่มทั้งหมด และแกนตั้งเป็นประเภทของเครื่องกำเนิดเลขสุ่มที่สามารถสร้างประชากรเริ่มต้นที่มีความหลากหลายมากที่สุด เมื่อเปรียบเทียบกับประชากรเริ่มต้นจากเครื่องกำเนิดเลขสุ่มทั้งหมด ซึ่งข้อมูลการทำงานของอัลกอริทึมแบบเมตาฮิวริสติกจะถูกดำเนินการตามขั้นตอน ดังนี้

(a) ทำความสะอาดข้อมูลด้วยการตัดส่วนที่เป็นตัวอักษรออกจากข้อความประวัติ ให้อยู่ในรูปแบบตาราง 2 มิติ ที่เก็บข้อมูลการสะสมของค่าต่าง ๆ แล้วนำมาหารเฉลี่ยด้วยจำนวนครั้งการทดลองทั้งหมด 30 ครั้ง

(b) สร้าง Dataframe ทั้งหมด 4 ตารางที่ใช้วิเคราะห์ค่าเหมาะสมโดยรวม ค่าความหลากหลาย ค่าการสำรวจ และเวลาที่ใช้ในการทำงาน โดยจำแนกเครื่องกำเนิดเลขสุ่มชนิดต่างกันทั้งหมด 4 ชนิด

(c) สร้างแผนภาพที่ใช้วิเคราะห์ข้อมูลค่าทั้ง 4 ชนิดของแต่ละกรณีทดสอบ เปรียบเทียบกันระหว่างชนิดของเครื่องกำเนิดเลขสุ่ม

(d) สร้างแผนภาพที่ใช้วิเคราะห์ข้อมูล (รูปที่ 3.12) ระหว่างค่าเหมาะสมที่ดีที่สุดในรอบการทำงานสุดท้าย และค่าความหลากหลายในรอบแรก เพื่อตรวจสอบสันนิษฐานที่เกิดขึ้นก่อนการทดลอง โดยค่าเหมาะสมที่ดีที่สุดในรอบสุดท้ายของปัญหา Max-cut จะพิจารณาเครื่องกำเนิดเลขสุ่มที่ให้ค่าเหมาะสมที่มากที่สุด แสดงถึงการสร้างจุดตัดจำนวนมาก และสำหรับปัญหาอื่นจะพิจารณาเครื่องกำเนิดเลขสุ่มที่ให้ค่าเหมาะสมน้อยที่สุดที่เป็นตัวแทนของระยะทางที่สั้นที่สุดในการแก้ไขปัญหา TSP และ ปัญหา CEC2021



รูปที่ 3.12 ภาพกระบวนการทำความสะอาด การวิเคราะห์ข้อมูล และตัวอย่างแผนภาพที่เกิดขึ้นจากการทดลอง

3.4.2 การวิเคราะห์ค่าสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปร (Pearson correlation coefficient analysis)

เนื่องจากผู้จัดทำต้องการทราบความสัมพันธ์ของชนิดเครื่องกำเนิดเลขสุ่ม และกระบวนการการทำงานของอัลกอริทึมแบบเมตาฮิวริสติก เพื่อใช้สรุปผลการทดลองและยืนยันสมมติฐาน จึงมีการนำข้อมูลการทดลองทั้งหมดมาวิเคราะห์ความสัมพันธ์ระหว่างตัวแปร ได้แก่ ชนิดของเครื่องกำเนิดเลขสุ่ม ค่าเหมาะสมในรอบสุดท้าย ค่าเหมาะสมในรอบแรก และความหลากหลายของประชากร โดยมีขั้นตอนจัดการข้อมูล ดังนี้

(a) ทำความสะอาดข้อมูลด้วยการตัดส่วนที่เป็นตัวอักษรออกจากข้อความประวัติ ให้อยู่ในรูปแบบตาราง 2 มิติ ที่เก็บข้อมูลการสะสมของค่าต่าง ๆ แล้วนำมาหารเฉลี่ยด้วยจำนวนครั้งการทดลองทั้งหมด 30 ครั้ง

(b) จัดทำ Dataframe เก็บข้อมูลค่าเฉลี่ยของตัวแปรที่สนใจศึกษาทั้งหมด โดยจะเฉลี่ยค่าตัวแปรตามกรณีทดสอบ และชนิดเครื่องกำเนิดเลขสุ่มที่ใช้สร้างประชากรเริ่มต้น ตารางที่ได้จะมีจำนวนแถวเท่ากับผลคูณระหว่างจำนวนอัลกอริทึม 3 ชนิด จำนวนกรณีทดสอบรวมทุกปัญหา 20 กรณี และจำนวนชนิดเครื่องกำเนิดเลขสุ่มทั้งหมด 4 ชนิด รวมจำนวนแถวทั้งหมด 240 แถว

(c) ทำการเข้ารหัสข้อมูลชนิดเครื่องกำเนิดเลขสุ่มให้กลายเป็นตัวเลขจำนวนเต็ม โดยกำหนดให้ PRNG แทนด้วยเลข 0 และ QRNG แทนด้วยเลข 1 และหาค่าสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปรที่สนใจด้วยคำสั่ง Dataframe.corr()

(d) สร้างแผนภาพความร้อนด้วยคำสั่งจากไลบรารี Seaborn โดยใส่ค่าสัมประสิทธิ์สหสัมพันธ์ลงในแผนภาพ

3.4.3 การวิเคราะห์ผลกระทบของเครื่องกำเนิดเลขสุ่มและคุณภาพประชากรด้วยการทดสอบฟรีดแมน (Friedman test)

การทดสอบฟรีดแมน (Friedman test) เป็นการทดสอบทางสถิติทำที่การวิเคราะห์ความแตกต่างของสิ่งที่สนใจด้วยการจัดลำดับของค่าที่เกี่ยวข้องกับการทดสอบ สามารถแสดงผลกระทบการเปลี่ยนวิธีการสุ่มประชากรเริ่มต้นส่งผลต่อคุณภาพประชากรสุดท้ายได้แตกต่างอย่างมีนัยสำคัญหรือไม่ และสามารถระบุลำดับของผลกระทบได้ [5], [30] ซึ่งมีการจัดการข้อมูลเพื่อทดสอบฟรีดแมนมีขั้นตอน และตัวอย่าง (ตารางที่ 3.1) ดังนี้

(a) ทำความสะอาดข้อมูลด้วยการตัดส่วนที่เป็นตัวอักษรออกจากข้อความประวัติ และเก็บเฉพาะค่าเหมาะสมโดยรวมรอบสุดท้ายทั้งหมด 30 ค่าที่เกิดจากการทดลอง 1 กรณีทดสอบต่อเครื่องกำเนิดเลขสุ่ม 1 ชนิด

(b) นำค่าเหมาะสมโดยรวมรอบสุดท้ายมาพิจารณาหาค่าเฉลี่ย (mean) ค่าความแปรปรวน (var) และค่าที่ดีที่สุด (best) ซึ่งหากเป็นกรณีปัญหา Max-cut ค่าทีเหมาะสมโดยรวมที่ดีที่สุดจะเป็นค่าเหมาะสมที่มากที่สุดจากจำนวนค่าเหมาะสมทั้งหมด 30 ค่า สำหรับปัญหา TSP และ CEC2021 ค่าทีเหมาะสมโดยรวมที่ดีที่สุดจะเป็นค่าเหมาะสมที่น้อยที่สุดจากจำนวนค่าเหมาะสมทั้งหมด 30 ค่า ตามวัตถุประสงค์ของการแก้ปัญหาแต่ละประเภท

(c) จัดทำ Dataframe เก็บข้อมูลของตัวแปรที่สนใจศึกษาทั้งหมด โดยแต่ละแถวของตารางจะมีค่าเฉลี่ย ค่าความแปรปรวน และค่าที่ดีที่สุด จำแนกตามกรณีทดสอบและเครื่องกำเนิดเลขสุ่มที่ใช้สร้างประชากรเริ่มต้น ตารางที่ได้จะมีจำนวนแถวเท่ากับผลคูณระหว่างจำนวนอัลกอริทึม 3 อัลกอริทึม จำนวนกรณีทดสอบรวมทุกปัญหา 20 กรณี และจำนวนชนิดเครื่องกำเนิดเลขสุ่มทั้งหมด 4 ชนิด รวมจำนวนแถวทั้งหมด 240 แถว

(d) ทำการเรียงลำดับค่าที่ดีที่สุด (rank_best) ลำดับค่าเฉลี่ย (rank_mean) และลำดับค่าความแปรปรวน (rank_var) ของค่าเหมาะสมรอบสุดท้าย โดยที่ค่าที่น้อยกว่าจะได้ลำดับที่น้อยกว่า ยกเว้นปัญหา Max-cut จะพิจารณาค่าเฉลี่ย และค่าที่ดีที่สุดที่มากที่สุดเป็นลำดับที่น้อยกว่า

(e) นำค่าลำดับทั้งหมดมาหาค่าเฉลี่ย (MOF, mean of ranks) จัดลำดับค่าเฉลี่ยระหว่างเครื่องกำเนิดเลขสุ่มภายในกรณีทดสอบเดียวกัน (rank_MOF) และจัดรูปแบบข้อมูลให้มีขนาด 2 มิติ เพื่อเข้าสู่กระบวนการทดสอบฟรีดแมนโดยกำหนดจะยอมรับสมมติฐานหลัก เมื่อค่า p-value มากกว่า 0.05 คือ ชนิดของเครื่องกำเนิดเลขสุ่มที่แตกต่างกันไม่ได้ส่งผลต่อคุณภาพประชากรสุดท้าย และจะยอมรับสมมติฐานทางเลือก เมื่อค่า p-value น้อยกว่า 0.05 คือ ชนิดของเครื่องกำเนิดเลขสุ่มอย่างน้อย 1 ชนิด ส่งผลต่อคุณภาพประชากรสุดท้ายได้มีความแตกต่างจากเครื่องกำเนิดสุ่มชนิดอื่น ๆ

ตารางที่ 3.1 ตัวอย่างข้อมูลและผลลัพธ์จากการจัดการข้อมูลก่อนการทดสอบฟรีดแมน

Algorithm	Testcase	RNG name	best	mean	var	rank_best	rank_mean	rank_var	MOF	rank_MOF
ABC	F102021	BeamSplitter	478.72	480.73	1.60	1	3	2	2.00	1
ABC	F102021	IBM	478.77	480.53	1.85	2	1	3	2.00	1
ABC	F102021	MT19937	479.15	480.57	0.51	4	2	1	2.33	2
ABC	F102021	QuasiRandom	478.81	493.56	187.31	3	4	4	3.67	3
PSO	St70	BeamSplitter	2748.13	2847.42	1482.56	2	2	2	2.00	2
PSO	St70	IBM	2750.44	2861.97	2216.84	3	4	3	2.00	4
PSO	St70	MT19937	2761.53	2841.06	2283.30	4	1	4	2.33	3
PSO	St70	QuasiRandom	2745.13	2854.40	1359.41	1	3	1	3.67	1

(f) นำค่าลำดับเฉลี่ย (rank_MOF) มาเฉลี่ยระหว่างกรณีทดสอบ เพื่อให้ได้ลำดับเฉลี่ยที่สื่อถึงคุณภาพประชากรสุดท้ายจากเครื่องกำเนิดเลขสุ่มชนิดต่าง ๆ ในระดับอัลกอริทึมและปัญหาที่สนใจ โดยนำเสนอในรูปแบบแผนภาพความร้อน

บทที่ 4 ผลการดำเนินงาน

การวิจัยครั้งนี้ผู้จัดทำจะออกแบบการทดลองตามวัตถุประสงค์ที่จะศึกษาผลกระทบของการใช้เลขสุ่มควอนตัมต่อประสิทธิภาพของการทำงานของอัลกอริทึมแบบเมตาฮิวริสติก การทดลองเพื่อพิสูจน์สมมติฐานจึงสามารถจำแนกได้เป็น 2 ประเด็น ได้แก่

(a) การทดลองวัดคุณสมบัติความสุ่มสุ่มของเครื่องกำเนิดเลขสุ่มแต่ละรูปแบบ ได้แก่ เครื่องกำเนิดเลขสุ่มควอนตัม และเครื่องกำเนิดเลขสุ่มเทียมที่ได้จากการคำนวณค่าที่ไม่สามารถคาดการณ์โดยมนุษย์ได้หากไม่ทราบ seed ผลการทดลองความสุ่มระดับบิตด้วยเครื่องมือทดสอบสถิติ NIST STS [24] คาดการณ์ว่าเครื่องกำเนิดเลขสุ่มควอนตัมควรมีจำนวนกรณีทดสอบผ่านมากกว่าเครื่องกำเนิดเลขสุ่มเทียมที่ออกแบบมาเพื่อหลีกเลี่ยงกรณีทดสอบที่มนุษย์รู้จัก

(b) การทดลองการใช้ QRNG ในขั้นตอนการสร้างประชากรเริ่มต้นส่งผลดีขึ้นต่อประสิทธิภาพของการทำงานเมตาฮิวริสติกอัลกอริทึม เมื่อพิจารณาจำนวนรอบที่เมตาฮิวริสติกอัลกอริทึมทำงาน ความหลากหลายของประชากร และค่าเหมาะสมของการทำงานรอบสุดท้าย

4.1 ผลการทดสอบคุณสมบัติความสุ่มสุ่มของเครื่องกำเนิดเลขสุ่ม

คณะผู้จัดทำได้นำ NIST STS มาใช้ทดสอบเลขสุ่ม เนื่องจากความนิยมใช้เครื่องมือในการวัดความสุ่มของเลขสุ่มของเครื่องกำเนิดเลขสุ่ม เอกสารประกอบเครื่องมือมีรายละเอียด และตัวอย่างการทำงานของกรณีทดสอบชัดเจน โดยในการทดสอบความสุ่มจากเครื่องกำเนิดเลขสุ่มแต่ละชนิด คณะผู้จัดทำได้กำหนดตัวแปรควบคุมให้คล้ายกับงานศึกษาในอดีต [31] ดังนี้

- ขนาดของ bit sequence ที่ใช้ทดสอบ เลือกใช้ขนาด 2^{20} บิต เพราะเป็นขนาดที่เลขสุ่มขนาด 32 บิตสามารถเรียงต่อกันโดยไม่ถูกตัดออกบางส่วน และเป็นขนาดที่ใกล้เคียงตัวอย่างที่เอกสาร NIST STS ใช้งาน (10^6 บิต).
- จำนวน bit sequence ที่ใช้ทดสอบ เลือกใช้จำนวน 100 บิต เพราะเป็นค่าที่เหมาะสมแก่ระดับนัยสำคัญ 0.01 ที่ถูกกำหนดไว้ในกรณีทดสอบของเครื่องมือ
- การใช้กรณีทดสอบเลขสุ่มทุกกรณีที่ NIST STS รองรับ จำนวนทั้งหมด 15 กรณีทดสอบ และมีกรณีทดสอบย่อยจำนวนทั้งหมด 188 กรณี

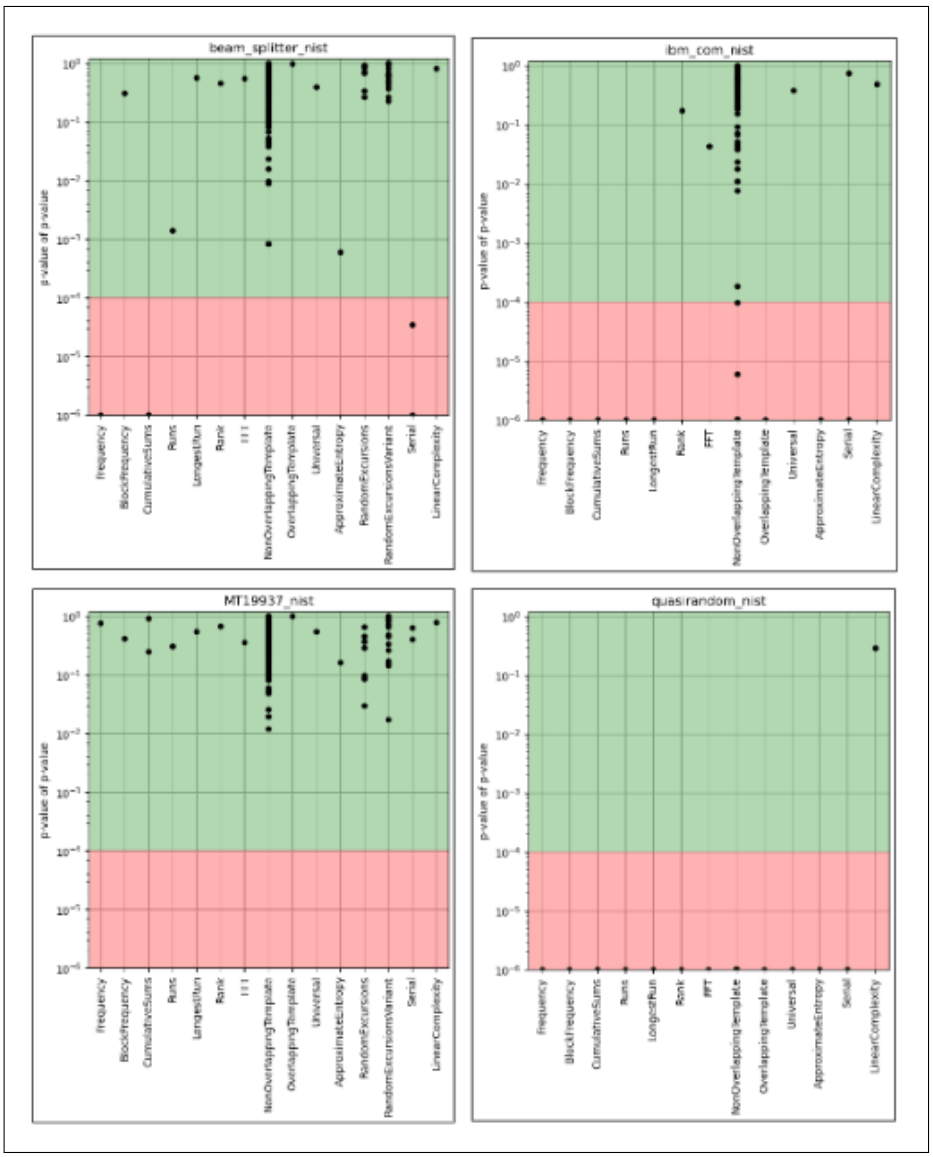
การพิจารณาความสุ่มที่เกิดขึ้นจาก NIST STS สามารถวิเคราะห์ได้โดยใช้ค่า p-value และสัดส่วนการทดสอบผ่านเปรียบเทียบกับการทดสอบทั้งหมด (pass ratio) [32] โดยมีรายละเอียด ดังนี้

(a) ผลการทดสอบ NIST STS โดยเปรียบเทียบจากค่า p-value of p-value

การตีความค่า p-value ของแต่ละ sequence ในกรณีทดสอบ จะถูกตีความได้โดยการเปรียบเทียบกับระดับนัยสำคัญ 0.01

- หากค่า p-value ≥ 0.01 สามารถตีความได้ว่า sequence เลขสุ่มมีความสุ่มที่ระดับความเชื่อมั่น 0.99 (Pass sequence)
- หากค่า p-value < 0.01 สามารถตีความได้ว่า sequence เลขสุ่มมีความไม่สุ่ม (non-randomness) ที่ระดับความเชื่อมั่น 0.99 (Fail sequence)

และการตีความค่า p-value ของแต่ละกรณีทดสอบ จะได้จากการคำนวณค่า p-value of p-value จาก sequence เลขสุ่มที่ถูกทดสอบ โดยใช้ฟังก์ชัน chi-squared เพื่อตรวจสอบการกระจายตัวของค่า p-value หลังจากนั้น นำค่า p-value of p-value ของแต่ละกรณีทดสอบ มาเปรียบเทียบกับระดับนัยสำคัญที่ 0.0001 (รูปที่ 4.1)



รูปที่ 4.1 ค่า p-value of p-value ที่ได้จากการทดสอบความสุ่มด้วย NIST STS

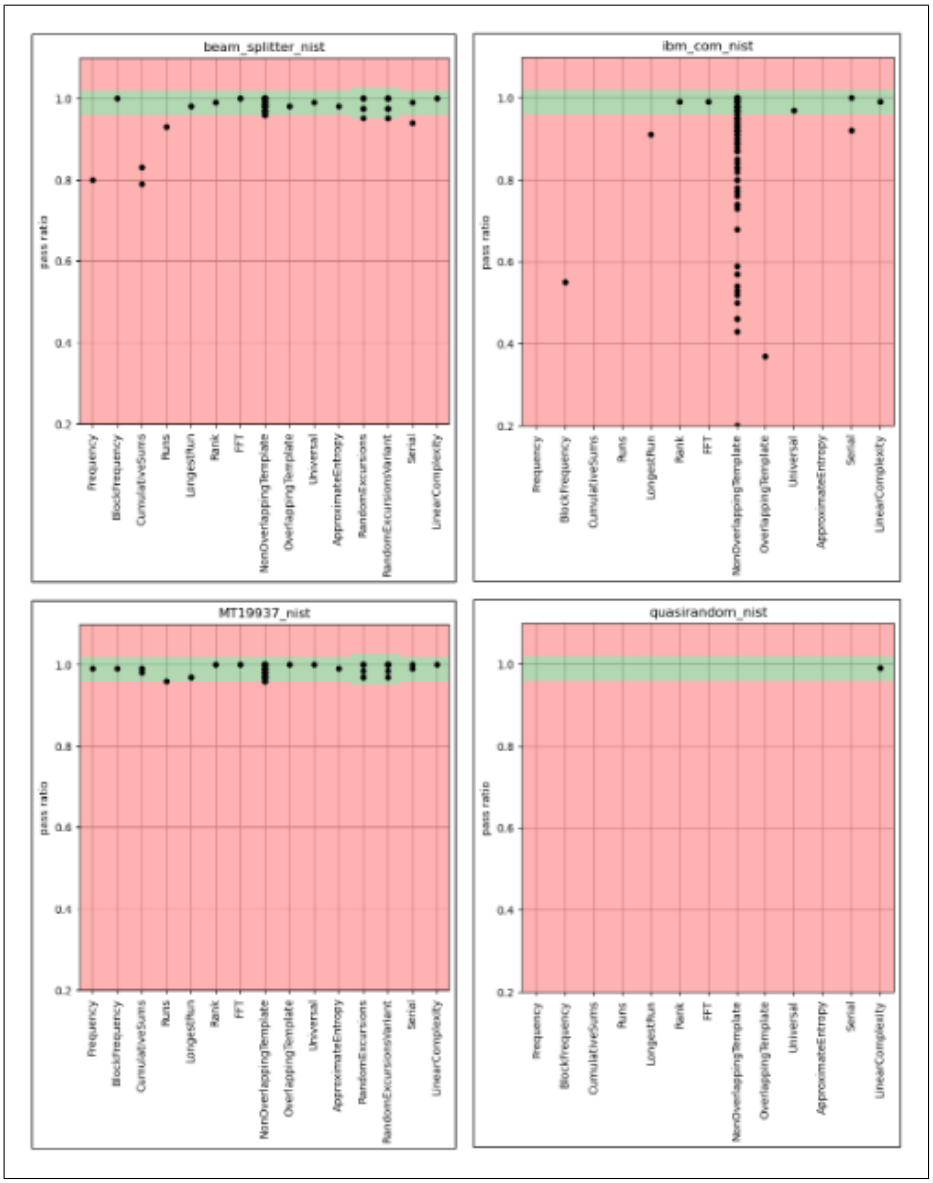
ตารางที่ 4.1 ตารางเปรียบจำนวนค่า p-value of p-value ในช่วงต่าง ๆ จากการทดสอบด้วย NIST STS

	MT19937	IBM quantum circuit RNG	Beam splitter RNG	Quasirandom (Halton)
Pass test count (p-value of p-value $\geq$ 0.0001)	188	86	183	1
Fail test count (p-value of p-value $<$ 0.0001)	0	102	5	187

หลังจากนั้น ค่า p-value ของแต่ละกรณีทดสอบจะต้องใช้ค่า pass ratio ประกอบ จึงจะสามารถตีความผลได้

(b) ผลการทดสอบ NIST STS โดยเปรียบเทียบจากค่า pass ratio และ p-value การตีความค่า pass ratio สามารถตีความได้โดยการเปรียบเทียบกับช่วงความเชื่อมั่น (confidence interval) ที่ถูกนิยามด้วย $(1-\alpha) \pm 3\sqrt{\frac{\alpha(1-\alpha)}{k}}$; โดยกำหนดระดับนัยสำคัญ 0.01 และต้องใช้ค่า p-value of p-value เพื่อสรุปผลคุณสมบัติความสุ่มของเลขสุ่ม (รูปที่ 4.2)

- หากค่า pass ratio อยู่ในช่วงความเชื่อมั่น และค่า p-value of p-value ของกรณีทดสอบ ≥ 0.0001 สามารถตีความได้ว่าเลขสุ่มผ่านกรณีทดสอบ หรือมีความสุ่มที่ระดับความเชื่อมั่น 0.99 (Pass)
- หากค่า pass ratio หรือ ค่า p-value ไม่ตรงตามเงื่อนไขแรก สามารถตีความได้ว่าเลขสุ่มไม่ผ่านกรณีทดสอบ หรือมีความไม่สุ่มที่ระดับความเชื่อมั่น 0.99 (Fail)



รูปที่ 4.2 ค่า pass ratio ที่ได้จากการทดสอบความสุ่มด้วย NIST STS

ตารางที่ 4.2 ตารางเปรียบเทียบจำนวนค่า pass ratio ในช่วงต่าง ๆ จากการทดสอบด้วย NIST STS

	MT19937	IBM quantum circuit RNG	Beam splitter RNG	Quasirandom (Halton)
Pass ratio fall in confidence interval	183	86	179	1
Pass ratio fall out of confidence interval	5	102	9	187

จากผลการทดลองจะสังเกตประเด็นต่าง ๆ ได้ ดังนี้

i. คุณสมบัติความสุ่มของเลขสุ่มจากอัลกอริทึม MT19937 มีความชัดเจนมากกว่าเครื่องกำเนิดเลขสุ่มชนิดอื่น จากจำนวน pass ratio และค่า p-value ที่ตรงตามเงื่อนไขมากกว่าเครื่องกำเนิดเลขสุ่มชนิดอื่น

ii. เครื่องกำเนิดเลขสุ่มเชิงควอนตัมผ่านกรณีได้จำนวนมากกว่าเครื่องกำเนิดเลขสุ่มจากวงจรควอนตัม IBM จึงมีคุณสมบัติความสุ่มที่ดีกว่า

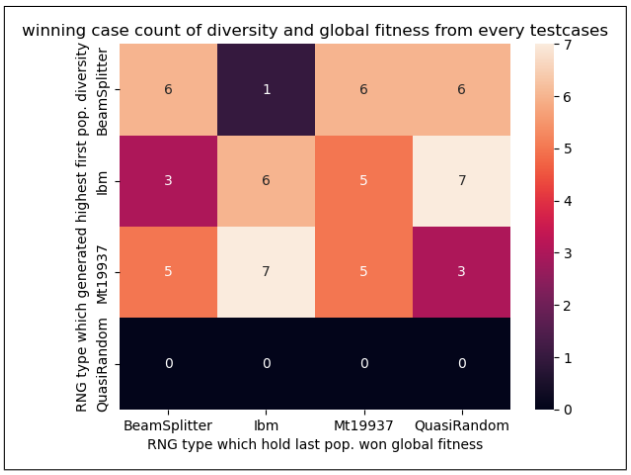
iii. เลขสุ่มที่ได้จากอัลกอริทึม Quasirandom ไม่ผ่านการทดสอบความสุ่มจากกรณีทดสอบส่วนใหญ่ สาเหตุเป็นเพราะเป็นค่าที่ถูกคำนวณเพื่อให้ผลลัพธ์มีการกระจายตัวแบบสม่ำเสมอเท่านั้น ไม่ได้มีการสลับลำดับเพื่อทำให้เป็นรูปแบบการสุ่ม แต่ผ่านกรณีทดสอบ Linear complexity

iv. สำหรับเลขสุ่มที่ได้จากเครื่องกำเนิดเลขสุ่มจากวงจรควอนตัม IBM และ อัลกอริทึม Quasirandom ไม่สามารถถูกทดสอบด้วยกรณีทดสอบ RandomExcursions และ RandomExcursionsVariant ได้ เนื่องจากเลขสุ่มมีคุณสมบัติความสุ่มน้อยเกินไป

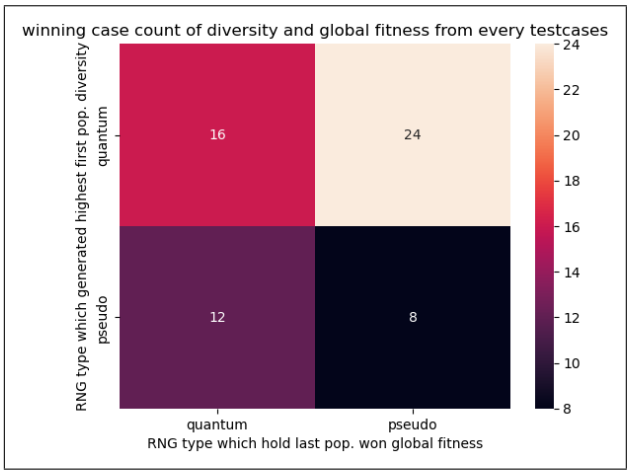
4.2 ผลการทดลองการทำงานของอัลกอริทึมแบบเมตาฮิวริสติก

4.2.1 ผลการวิเคราะห์การชนะของค่าเหมาะสมโดยรวม และความหลากหลายของประชากร

คณะผู้จัดทำพบว่าการสร้างประชากรเริ่มต้นด้วย QRNG ที่มีแหล่งที่มาจาก IBM ควอนตัมคอมพิวเตอร์ หรือการทดลองแสงเชิงควอนตัม สามารถทำให้อัลกอริทึมแบบเมตาฮิวริสติกหาคำตอบค่าเหมาะสมที่ดีที่สุดได้ดีกว่าการสร้างประชากรด้วย PRNG จากแหล่งที่มา ได้แก่ อัลกอริทึม MT19937 หรือ Quasirandom โดยการสร้างประชากรเริ่มต้นด้วย QRNG สามารถนำเสนอค่าความหลากหลายของประชากรเริ่มต้นมากกว่าการสร้างประชากรเริ่มต้นด้วย PRNG 40 กรณีทดสอบ จากทั้งหมด 60 กรณีทดสอบ คิดเป็นร้อยละ 66.67 และ หากพิจารณาค่าเหมาะสมโดยรวม ณ รอบสุดท้าย การสร้างประชากรเริ่มต้นด้วย QRNG จะได้ค่าเหมาะสมโดยรวมรอบสุดท้ายคุณภาพดีกว่า PRNG จำนวน 28 กรณีทดสอบ จากทั้งหมด 60 กรณีทดสอบ คิดเป็นร้อยละ 46.67% ซึ่งสามารถสังเกตจากแผนภาพความร้อน (รูปที่ 4.3 และ 4.4)



รูปที่ 4.3 แผนภาพความร้อนแสดงจำนวนการชนะของค่าที่เกิดขึ้นจากการสร้างประชากรเริ่มต้นด้วยเครื่องกำเนิดเลขสุ่มชนิดต่าง ๆ



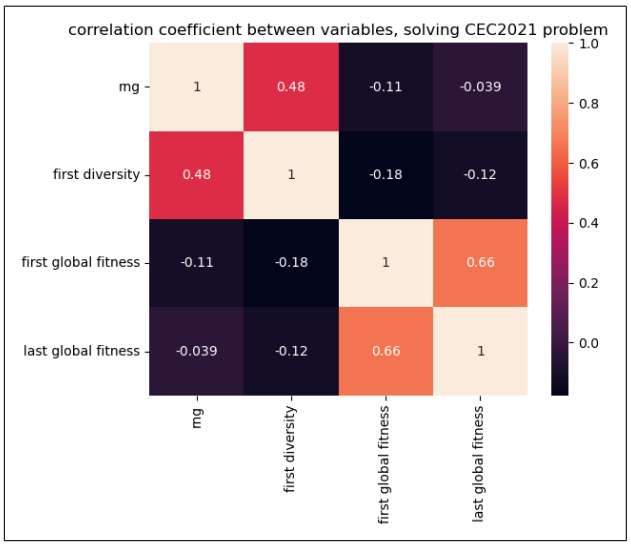
รูปที่ 4.4 แผนภาพความร้อนแสดงจำนวนการชนะของค่าที่เกิดขึ้นจากการสร้างประชากรเริ่มต้นด้วยเครื่องกำเนิดเลขสุ่มประเภทต่าง ๆ

หลังจากการสังเกตคณะผู้จัดทำพบว่า สมมติฐานของผู้จัดทำได้ตั้งไว้ก่อนเริ่มการทดลองไม่ตรงตามที่คาดหวังทั้งหมด เมื่อพิจารณาจำนวนการชนะของค่าความหลากหลาย การสร้างประชากรเริ่มต้นด้วย QRNG ทำให้การกระจายตัวของประชากรเริ่มต้นมากกว่าประชากรเริ่มต้นที่ได้จาก PRNG แต่การสร้างประชากรเริ่มต้นด้วย QRNG ไม่ทำให้ค่าเหมาะสมโดยรวมของประชากรรุ่นสุดท้ายดีกว่าประชากรเริ่มต้นที่ได้จาก PRNG อย่างชัดเจน และพบว่า มีกรณีที่เครื่องกำเนิดเลขสุ่ม Quasirandom ไม่ได้สร้างประชากรเริ่มต้นที่มีความหลากหลายมากที่สุด แต่ประชากรชุดเดิมสามารถเป็นคำตอบที่ดีที่สุดได้กว่า 16 กรณีทดสอบ หรือกรณีที่เครื่องกำเนิดเลขสุ่มชนิดใด ๆ ที่สามารถสร้างประชากรเริ่มต้นที่มีความหลากหลายมากที่สุด แต่ประชากรกลุ่มเดิมไม่สามารถได้คำตอบที่ดีที่สุดเมื่อเทียบกับประชากรที่ได้จากเครื่องกำเนิดเลขสุ่มชนิดอื่น ซึ่งหักล้างข้อสันนิษฐานการสร้างประชากรเริ่มต้นที่มีความหลากหลายจะส่งผลต่อค่าเหมาะสมของประชากรที่ดีในรอบสุดท้าย

4.2.2 ผลการวิเคราะห์ค่าสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปร

เมื่อชนิดของเครื่องกำเนิดเลขสุ่มถูกเข้ารหัสเป็นตัวเลขจำนวนเต็ม โดยที่ 0 และ 1 แทน PRNG และ QRNG ตามลำดับ ทำให้สามารถสร้างแผนภาพความร้อน (รูปที่ 4.5, 4.6, 4.7) ที่แสดงถึงค่าสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปรต่าง ๆ โดยจำแนกตามประเภทปัญหาที่ทำการศึกษา ดังนี้

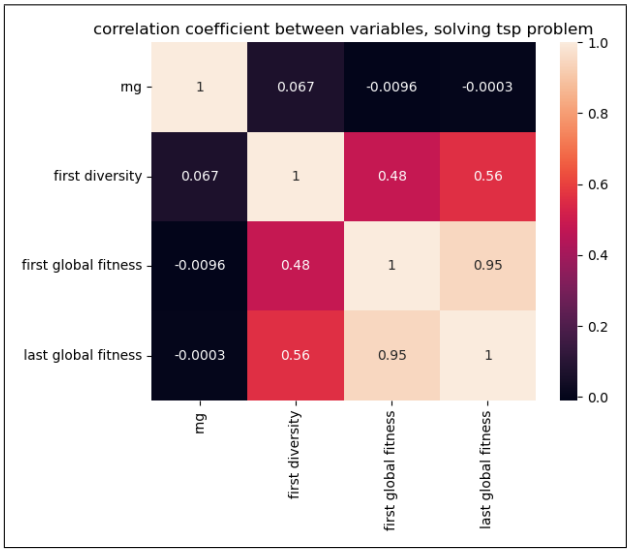
- ปัญหา CEC2021



รูปที่ 4.5 แผนภาพความร้อนแสดงสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปรของอัลกอริทึมแบบเมตาฮิวริสติกที่ใช้แก้ไขปัญหา CEC2021

จากแผนภาพ (รูปที่ 4.5) พบว่า ในการแก้ไขปัญหา CEC2021 ชนิดของเครื่องกำเนิดเลขสุ่มส่งผลต่อความหลากหลายของประชากรเริ่มต้น โดยการเปลี่ยนแปลงชนิดเครื่องกำเนิดเลขสุ่มจาก PRNG เป็น QRNG มีแนวโน้มทำให้ความหลากหลายของประชากรมากขึ้นด้วยค่าสัมประสิทธิ์สหสัมพันธ์ 0.48 และประชากรเริ่มต้นที่มีค่าเหมาะสมสูงมีแนวโน้มที่จะทำให้ได้ค่าเหมาะสมจากประชากรสุดท้ายสูงด้วยค่าสัมประสิทธิ์สหสัมพันธ์ 0.66 แต่ไม่พบความเกี่ยวข้องที่ชัดเจนระหว่างชนิดของเครื่องกำเนิดเลขสุ่ม และค่าเหมาะสมโดยรวมของประชากรเริ่มต้น หรือค่าเหมาะสมโดยรวมของประชากรรอบสุดท้าย

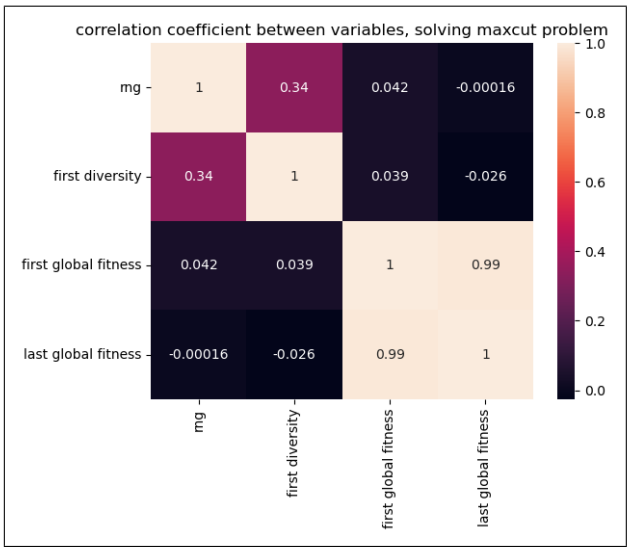
- ปัญหา TSP



รูปที่ 4.6 แผนภาพความร้อนแสดงสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปรของอัลกอริทึมแบบเมตาฮิวริสติกที่ใช้แก้ไขปัญหา TSP

จากแผนภาพ (รูปที่ 4.6) พบว่า ในการแก้ไขปัญหา TSP ความหลากหลายของประชากรเริ่มต้น ค่าเหมาะสมโดยรวมของประชากรเริ่มต้น และค่าเหมาะสมโดยรวมของประชากรสุดท้ายมีความสัมพันธ์ไปในทิศทางเดียวกันจากค่าสัมประสิทธิ์สหสัมพันธ์ที่มากกว่า 0.4 ทั้งหมด แต่ชนิด ของเครื่องกำเนิดเลขสุ่มไม่มีความสัมพันธ์ที่ชัดเจนต่อตัวแปรใด ๆ

- ปัญหา Max-cut



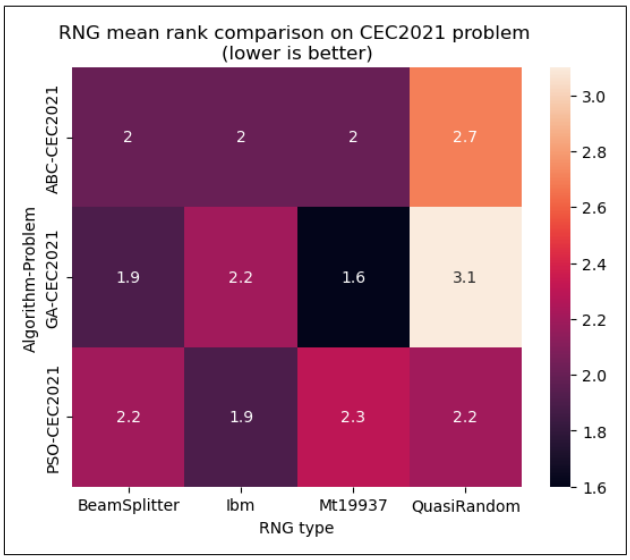
รูปที่ 4.7 แผนภาพความร้อนแสดงสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปรของอัลกอริทึมแบบเมตาฮิวริสติกที่ใช้แก้ไขปัญหา Max-cut

จากแผนภาพ (รูปที่ 4.7) พบว่า ในการแก้ไขปัญหา Max-cut มีค่าสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปรคล้ายกับการแก้ไขปัญหา CEC2021 โดยชนิดของเครื่องกำเนิดเลขสุ่มส่งผลต่อความหลากหลายของประชากรเริ่มต้น โดยการเปลี่ยนแปลงชนิดเครื่องกำเนิดเลขสุ่มจาก PRNG เป็น QRNG มีแนวโน้มทำให้ความหลากหลายของประชากรมากขึ้นด้วยค่าสัมประสิทธิ์สหสัมพันธ์ 0.34 และประชากรเริ่มต้นที่มีค่าเหมาะสมสูงมีแนวโน้มที่จะทำให้ได้ค่าเหมาะสมจากประชากรสุดท้ายสูงด้วยค่าสัมประสิทธิ์สหสัมพันธ์ 0.99 แต่ไม่พบความเกี่ยวข้องที่ชัดเจนระหว่างชนิดของเครื่องกำเนิดเลขสุ่ม และค่าเหมาะสมโดยรวมของประชากรเริ่มต้น หรือค่าเหมาะสมโดยรวมของประชากรรอบสุดท้าย

4.2.3 ผลการวิเคราะห์ผลกระทบของเครื่องกำเนิดเลขสุ่มและคุณภาพประชากรด้วยการทดสอบฟรีดแมน

การวิเคราะห์ผลกระทบของเครื่องกำเนิดเลขสุ่มและคุณภาพประชากร จะถูกจัดการและวิเคราะห์ด้วยการทดสอบฟรีดแมน โดยผลลัพธ์ที่ได้จากการทดสอบจะอยู่ในรูปแบบลำดับเฉลี่ยของเครื่องกำเนิดเลขสุ่มแต่ละชนิด ที่แสดงถึงคุณภาพค่าเหมาะสมโดยรวมของประชากรสุดท้าย 3 ด้าน ได้แก่ ความแปรปรวนของค่าเหมาะสมที่ควรมีค่าต่ำซึ่งแสดงถึงคำตอบที่คงที่ในทุกครั้งของการทดลอง ค่าเฉลี่ยของค่าเหมาะสมโดยรวม และค่าเหมาะสมโดยรวมที่เหมาะสมที่สุดในทุกครั้งของการทดลอง ซึ่งสามารถรายงานลำดับทั้งหมดจำแนกตามชนิดปัญหาด้วยแผนภาพความร้อน และอธิปรายผลโดยการสังเกต p-value (ตารางที่ 4.3) ดังนี้

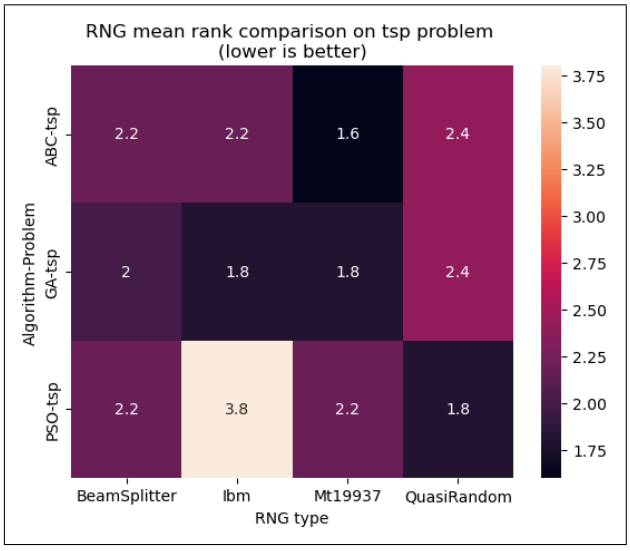
- ปัญหา CEC2021



รูปที่ 4.8 แผนภาพความร้อนแสดงลำดับเฉลี่ยของประชากรสุดท้ายจากเครื่องกำเนิดเลขสุ่มชนิดต่าง ๆ สำหรับแก้ไขปัญหา CEC2021

จากแผนภาพ (รูปที่ 4.8) พบว่าการแก้ไขปัญหา CEC2021 โดยใช้อัลกอริทึม ABC และ PSO ลำดับเฉลี่ยประชากรสุดท้ายของเครื่องกำเนิดเลขสุ่มแต่ละชนิดไม่แตกต่างกัน ปฏิเสธสมมติฐานหลักด้วย p-value 0.504776 และ 0.908139 ตามลำดับ แต่สำหรับอัลกอริทึม GA มีลำดับเฉลี่ยประชากรสุดท้ายของเครื่องกำเนิดเลขสุ่มแต่ละชนิดที่แตกต่างกันอย่างชัดเจน โดยยอมรับสมมติฐานหลักด้วย p-value 0.007718 แสดงถึงความอ่อนไหวต่อการเปลี่ยนชนิดเครื่องกำเนิดเลขสุ่ม โดยหากพิจารณาลำดับเฉลี่ยเป็น QRNG และ PRNG จะได้ลำดับเฉลี่ยของ QRNG ที่ 2.05 น้อยกว่าลำดับเฉลี่ยของ PRNG ที่ 2.35

- ปัญหา TSP

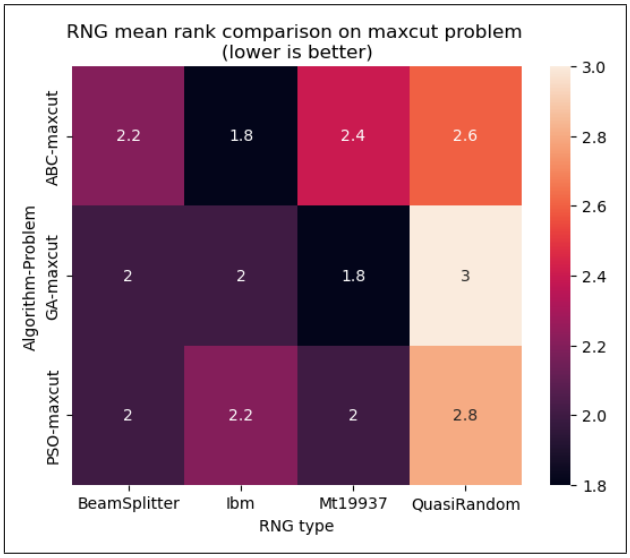


รูปที่ 4.9 แผนภาพความร้อนแสดงลำดับเฉลี่ยของประชากรสุดท้ายจากเครื่องกำเนิดเลขสุ่มชนิดต่าง ๆ สำหรับแก้ไขปัญหา TSP

จากแผนภาพ (รูปที่ 4.9) พบว่าการแก้ไขปัญหา TSP โดยใช้อัลกอริทึม ABC และ GA ลำดับเฉลี่ยประชากรสุดท้ายของเครื่องกำเนิดเลขสุ่มแต่ละชนิดไม่แตกต่างกัน ปฏิเสธสมมติฐานหลักด้วย p-value 0.558686 และ 0.801252 ตามลำดับ แต่อัลกอริทึม PSO มีลำดับเฉลี่ยประชากรสุดท้ายของเครื่องกำเนิดเลขสุ่มแต่ละชนิดที่แตกต่างกัน สังเกตจาก

Ibm QRNG ที่ได้ลำดับเฉลี่ยที่ 3.8 แตกต่างจากเครื่องกำเนิดเลขสุ่มทั้งหมด แต่ไม่มีนัยสำคัญที่จะปฏิเสธสมมติฐานหลักด้วย p-value 0.069391

- ปัญหา Max-cut



รูปที่ 4.10 แผนภาพความร้อนแสดงลำดับเฉลี่ยของประชากรสุดท้ายจากเครื่องกำเนิดเลขสุ่มชนิดต่าง ๆ สำหรับแก้ไขปัญหา Max-cut

จากแผนภาพ (รูปที่ 4.10) พบว่าการแก้ไขปัญหา Maxcut โดยใช้อัลกอริทึม ABC, GA และ PSO ลำดับเฉลี่ยประชากรสุดท้ายของเครื่องกำเนิดเลขสุ่มแต่ละชนิดไม่แตกต่างกัน ปฏิเสธสมมติฐานหลักด้วย p-value 0.689656, 0.273183 และ 0.771043 ตามลำดับ

ตารางที่ 4.3 ตารางค่า p-value ของการทดสอบฟรีดแมนจำแนกตามชนิดของอัลกอริทึม และปัญหาที่ใช้ทดสอบ

Friedman test p-value		Algorithm		
		ABC (0.263240)	GA (0.002051)	PSO (0.864180)
Problem	CEC2021 (0.048844)	0.504776	0.007718	0.908139
	TSP (0.443881)	0.558686	0.801252	0.069391
	Max-cut (0.201258)	0.689656	0.273183	0.771043

นอกจากนี้ ค่า p-value ที่ได้จากการทดสอบฟรีดแมนยังสามารถวิเคราะห์ผลในระดับอัลกอริทึม หรือในระดับปัญหารายย่อยได้ ซึ่งพบว่าประชากรสุดท้ายที่ได้จากอัลกอริทึม GA มีความอ่อนไหวต่อชนิดของเครื่องกำเนิดเลขสุ่มอย่างมีนัยสำคัญ และประชากรสุดท้ายที่แก้ไขปัญหา CEC2021 มีความอ่อนไหวต่อชนิดของเครื่องกำเนิดเลขสุ่มอย่างมีนัยสำคัญ

4.2.4 การอภิปรายผลการทดลองต่อสมมติฐานการวิจัย

1. จากผลการทดสอบคุณสมบัติความสุ่มของเลขสุ่มจากเครื่องกำเนิดเลขสุ่มแต่ละชนิดด้วย NIST STS พบว่า เครื่องกำเนิดเลขสุ่มที่มีจำนวนกรณีผ่านการทดสอบมากที่สุด คือ MT19937, เครื่องกำเนิดเลขสุ่มจาก IBM ควอนตัมคอมพิวเตอร์, เครื่องกำเนิดเลขสุ่มจากการทดลองแสงเชิงควอนตัม และ Quasirandom ตามลำดับ ซึ่งจะสังเกตได้ว่า QRNG ไม่ได้มีคุณสมบัติความสุ่มที่ดีมากกว่า PRNG ชนิด MT19937 ทำให้ข้อสมมติฐานการวิจัยข้อ 1 ไม่ถูกต้อง ทั้งนี้ เครื่องกำเนิดเลขสุ่ม MT19937 เป็นเครื่องกำเนิดเลขสุ่มที่ถูกออกแบบเพื่อหลีกเลี่ยงกรณีทดสอบในเครื่องมือ NIST STS อาจทำส่งผลให้ผ่านการทดสอบได้มากกว่าเครื่อง

กำเนิดเลขสุ่มชนิดอื่น และการวัดคุณสมบัติความสุ่มของตัวเลขอาจจะไม่สามารถวัดได้จริง เพราะสิ่งที่เครื่องมือ NIST STS และเครื่องมือวัดความสุ่มอื่น ๆ ทำการตรวจสอบ คือ รูปแบบที่ซ้ำเกิดขึ้นซ้ำกันในตัวเลข ซึ่งรูปแบบดังกล่าวเป็นรูปแบบที่มนุษย์รู้จักและกำหนดขึ้นมาเพื่อทดสอบ ไม่ได้เป็นความสุ่มที่แท้จริงที่มนุษย์ไม่รู้จัก

2. จากผลการวิเคราะห์การชนะของค่าเหมาะสมโดยรวม และความหลากหลายของประชากร พบว่ามีกรณีทดสอบที่เครื่องกำเนิดเลขสุ่มที่สามารถสร้างประชากรเริ่มต้นได้มีความหลากหลายมากที่สุด ไม่ได้ประชากรสุดท้ายที่มีค่าเหมาะสมโดยรวมดีที่สุดเมื่อเปรียบกับประชากรจากเครื่องกำเนิดเลขสุ่มชนิดอื่น ๆ และจากผลการวิเคราะห์ค่าสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปรพบว่า ค่าความหลากหลายของประชากรเริ่มต้นไม่ได้ส่งผลต่อค่าเหมาะสมโดยรวมของประชากรสุดท้ายในการแก้ไขปัญหา CEC2021 และ Max-cut แต่พบค่าสัมประสิทธิ์สหสัมพันธ์สื่อถึงความเกี่ยวข้องกันระหว่าง 2 ตัวแปรในการแก้ไขปัญหา TSP การสรุปสมมติฐานการวิจัยข้อที่ 2 จึงขึ้นอยู่กับประเภทของปัญหาที่ใช้พิจารณาสมมติฐาน

3. จากผลการวิเคราะห์การชนะของค่าเหมาะสมโดยรวม และความหลากหลายของประชากร พบว่า QRNG มีจำนวนการชนะของค่าความหลากหลายของประชากรเริ่มต้นมากกว่า แต่สำหรับค่าความเหมาะสมโดยรวมของประชากรสุดท้ายพบว่าประชากรที่ได้จาก PRNG มีจำนวนการชนะที่มากกว่า และจากผลการวิเคราะห์ค่าสัมประสิทธิ์สหสัมพันธ์ระหว่างตัวแปรพบว่าการเปลี่ยนแปลงชนิดเครื่องกำเนิดเลขสุ่มจาก PRNG เป็น QRNG มีสัมประสิทธิ์สหสัมพันธ์เชิงบวกกับค่าความหลากหลายของประชากรเริ่มต้นในปัญหา CEC2021 และ Max-cut แต่สิ่งดังกล่าวไม่เกิดขึ้นสำหรับการแก้ไขปัญหา TSP ทำให้การสรุปสมมติฐานการวิจัยข้อที่ 3 และ 4 จึงขึ้นอยู่กับประเภทของปัญหาที่ใช้พิจารณาสมมติฐาน ดังนี้

- สำหรับปัญหา CEC2021 และ Max-cut, การใช้ QRNG มีแนวโน้มที่จะสร้างประชากรเริ่มต้นที่มีความหลากหลายมากกว่า PRNG แต่ไม่มีแนวโน้มที่จะสร้างประชากรที่มีค่าเหมาะสมโดยรวมที่ดีกว่า PRNG
- สำหรับปัญหา TSP ไม่มีแนวโน้มที่ประเภทของเครื่องกำเนิดเลขสุ่มจะส่งผลต่อความหลากหลาย หรือค่าเหมาะสมโดยรวมของประชากรสุดท้าย

4. จากผลการวิเคราะห์ผลกระทบของชนิดเครื่องกำเนิดเลขสุ่มและคุณภาพประชากรด้วยการทดสอบฟรีดแมน พบว่าเฉพาะอัลกอริทึม GA และการแก้ไขปัญหา CEC2021 เท่านั้น ที่การเปลี่ยนแปลงชนิดเครื่องกำเนิดเลขสุ่มส่งผลต่อประชากรสุดท้ายอย่างมีนัยสำคัญ ซึ่งคณะผู้จัดทำคาดการณ์สาเหตุว่า กระบวนการของอัลกอริทึม GA เป็นสร้างคำตอบจากประชากรจำนวนน้อยต่อ 1 ครั้ง จึงทำให้คุณสมบัติของประชากรเริ่มต้นยังคงถูกรักษาไว้ตลอดการสร้างประชากรรุ่นต่อไป แต่เมื่อเปรียบเทียบกับกรณีอื่น ๆ ที่ใช้อัลกอริทึมที่มีหลักการของฝูง (Swarm-based) และปัญหา CEC2021 เป็นปัญหาที่ใช้เลขสุ่มโดยไม่มีข้อจำกัดในปริภูมิค้นหามากเมื่อเปรียบเทียบกับปัญหา TSP และ Max-cut จึงทำให้ใช้ตัวเลขสุ่มได้โดยตรงโดยไม่สูญเสียความสุ่มในปริภูมิค้นหาไป สำหรับอัลกอริทึมแบบเมตาฮิวริสติก หรือปัญหาชนิดอื่นที่อยู่ในการศึกษาครั้งนี้ ถึงแม้ว่าจะมีความแตกต่างของลำดับเฉลี่ยระหว่าง PRNG และ QRNG บ้าง เช่น ในปัญหา Max-cut ทุกอัลกอริทึมที่ใช้ QRNG มีลำดับเฉลี่ยภายในเครื่องกำเนิดเลขสุ่มประเภทเดียวกัน ที่ต่ำกว่าลำดับเฉลี่ยของ PRNG จริง แต่ไม่มีนัยสำคัญที่จะยอมรับได้ว่าการเปลี่ยนชนิดเครื่องกำเนิดเลขสุ่มส่งผลต่อประชากรสุดท้าย

บทที่ 5 บทสรุป

5.1 สรุปผลการดำเนินงาน

ในการศึกษาครั้งนี้ คณะผู้จัดทำได้ทำการทดลองการสร้างประชากรเริ่มต้นด้วยเครื่องกำเนิดเลขสุ่ม 2 ประเภท รวมทั้งหมด 4 ชนิด จำแนกเป็นเครื่องกำเนิดเลขสุ่มควอนตัม 2 ชนิด และเครื่องกำเนิดเลขสุ่มเทียม 2 ชนิด จากผลการศึกษาพบว่าเครื่องกำเนิดเลขสุ่มควอนตัมมีคุณสมบัติความสุ่มที่ด้อยกว่าเครื่องกำเนิดเลขสุ่มเทียมบางชนิด ซึ่งหลังจากการนำเครื่องกำเนิดเลขสุ่มทั้งหมดไปใช้งานสร้างประชากรเริ่มต้นให้กับอัลกอริทึมแบบเมตาฮิวริสติก 3 ชนิด ได้แก่ GA, PSO และ ABC เพื่อแก้ไขปัญหา 3 ชนิด ได้แก่ CEC2021, TSP และ ปัญหา Max-cut สามารถสังเกตค่าสัมประสิทธิ์สหสัมพันธ์พบว่า การสร้างประชากรเริ่มต้นด้วยเครื่องกำเนิดเลขสุ่มควอนตัมมีความสัมพันธ์เชิงบวกกับค่าความหลากหลายของประชากรเริ่มต้นในการแก้ไขปัญหา CEC2021 และปัญหา Max-cut โดยเมื่อพิจารณาปัญหาที่ใช้ทดสอบทั้งหมดจะไม่พบความสัมพันธ์ระหว่างประเภทของเครื่องกำเนิดเลขสุ่มต่อค่าเหมาะสมโดยรวมของประชากรสุดท้าย และ จากการสังเกตผลการทดสอบฟรีดแมน พบว่า การสร้างประชากรด้วยเครื่องกำเนิดเลขสุ่มที่แตกต่างกันส่งผลต่อคุณภาพของประชากรสุดท้ายของอัลกอริทึม GA ที่แก้ไขปัญหา CEC2021 ได้อย่างมีนัยสำคัญด้วยค่า p-value 0.007718 ซึ่งถ้าพิจารณาลำดับเฉลี่ยของคุณภาพประชากรสุดท้ายจะพบว่าเครื่องกำเนิดเลขสุ่มควอนตัมได้ลำดับน้อยกว่าเครื่องกำเนิดเลขสุ่มเทียม สำหรับอัลกอริทึมอื่น ๆ ที่แก้ปัญหาอื่น ๆ นอกเหนือจากอัลกอริทึม GA ที่แก้ไขปัญหา CEC2021 อาจจะมีลำดับเฉลี่ยของเครื่องกำเนิดเลขสุ่มควอนตัมน้อยกว่าลำดับเฉลี่ยของเครื่องกำเนิดเลขสุ่มเทียมได้ เช่น ปัญหา Max-cut แต่ผลการทดสอบฟรีดแมนยอมรับสมมติฐานหลักที่ $p\text{-value} > 0.05$ หรือ สรุปได้ว่าการเปลี่ยนชนิดของเครื่องกำเนิดเลขสุ่มไม่ได้ส่งผลต่อคุณภาพของประชากรสุดท้ายอย่างมีนัยสำคัญ

5.2 ปัญหาที่พบและการแก้ไข

ในภาคการศึกษาที่ 2 คณะผู้จัดทำได้เริ่มกระบวนการการทดลอง และพบปัญหา ดังนี้

5.2.1 ข้อจำกัดของอุปกรณ์การทดลอง

อุปกรณ์ที่ใช้ในการทดลองมีข้อจำกัดทางด้านเวลาที่ใช้ในการทำงานอัลกอริทึมแบบเมตาฮิวริสติก เนื่องจากการทดลองรูปแบบเก่าไม่ได้มีการกำหนดคุณสมบัติของอุปกรณ์ หรือระบบที่ใช้ทดลองที่ชัดเจน การทดลองครั้งแรกจึงใช้คอมพิวเตอร์พกพาที่มีหน่วยประมวลผล AMD Ryzen 4600H (6 cores) บนระบบปฏิบัติการ Microsoft Windows 11 ทำให้การทดลอง 1 อัลกอริทึมใช้เวลานาน 2-3 วันโดยประมาณ คณะผู้จัดทำจึงเปลี่ยนแปลงเครื่องมือ และระบบที่ใช้ทดสอบเป็นเซิร์ฟเวอร์ที่ใช้หน่วยประมวลผลที่มีหน่วยประมวลผล Intel Xeon E5-2698 v3 (16 cores) และใช้ระบบปฏิบัติการ Linux เพื่อให้การทดลองสามารถทำงานได้ตลอดเวลา และความรวดเร็วในการประมวลผล

5.2.2 ปัญหาการอ่านไฟล์ผิดพลาด

หลังจากการทดลองครั้งแรก คณะผู้จัดทำได้ทดลองอัลกอริทึม GA กับปัญหา TSP และพบว่าไม่มีผลลัพธ์ของการทดลองเมื่อสร้างประชากรเริ่มต้นด้วยเครื่องกำเนิดเลขสุ่มจาก IBM ควอนตัมคอมพิวเตอร์ และ ปัญหา Max-cut ในบางกรณีทดสอบ ซึ่งเกิดจากปัญหาคำสั่งสร้างไฟล์ และการอ่านชื่อไฟล์ที่มีตัวอักขระพิเศษ และปัญหาทั้งหมดถูกแก้ไขให้ทำงานได้อย่างถูกต้องแล้ว

5.2.3 ปัญหาความไม่แน่นอนของผลการทดลอง

หลังจากการทดลองดำเนินการสำเร็จทั้งหมด 3 อัลกอริทึม และ 3 ปัญหา คณะผู้จัดทำพบว่าผลของการทดลองมีการเปลี่ยนแปลงจากการทดลองอัลกอริทึมครั้งแรก ที่ทำการทดลองเฉพาะอัลกอริทึม GA แก้ไขปัญหา TSP โดยลำดับของค่าความหลากหลายในรอบแรก และค่าเหมาะสมของคำตอบที่ได้จากเครื่องกำเนิดเลขสุ่มชนิดต่าง ๆ มีการเปลี่ยนแปลง ซึ่งคาดว่าอาจจะเกิดจากจำนวนการทดลองซ้ำน้อยกว่าการดำเนินการแบบสุ่มภายในอัลกอริทึมทำงาน เช่น ความน่าจะเป็นของการกลายพันธุ์ใน

อัลกอริทึม GA ผู้จัดทำจึงตัดสินใจที่จะเพิ่มจำนวนการทดลองซ้ำจาก 10 ครั้งกลายเป็น 30 ครั้ง เพื่อลดผลกระทบการดำเนินการสุ่มภายในอัลกอริทึมแบบเมตาฮิวริสติกทั้งหมด

5.3 ข้อจำกัดและข้อเสนอแนะ

- ในการศึกษาครั้งนี้ มีเครื่องกำเนิดเลขสุ่มทั้งหมด 4 ชนิด แบ่งออกเป็นเครื่องกำเนิดเลขสุ่มควอนตัม 2 ชนิด และเครื่องกำเนิดเลขสุ่มเทียม 2 ชนิด ทำให้สิ่งที่ค้นพบอาจจะไม่เพียงพอต่อการสรุปให้ครอบคลุมเครื่องกำเนิดเลขสุ่มชนิดอื่น ๆ ที่ไม่ได้ถูกทดลองในการศึกษาครั้งนี้ ดังนั้น การเพิ่มจำนวนชนิดเครื่องกำเนิดเลขสุ่ม จำนวนปัญหาทดสอบ หรือ จำนวนอัลกอริทึมแบบเมตาฮิวริสติกสำหรับการทดลองจึงเป็นสิ่งที่จำเป็นอย่างยิ่งในการยืนยัน หรือปฏิเสธสมมติฐานการวิจัยที่ครอบคลุมการใช้งานทั้งหมด
- ในการตีความผลลัพธ์จากการทดลองบางส่วนในการทดสอบฟรีดแมนอาจจะมีความคลาดเคลื่อน และไม่ครบถ้วนสมบูรณ์ เนื่องจากคณะผู้จัดทำไม่สามารถใช้โปรแกรม IBM SPSS ได้ด้วยข้อจำกัดของงบประมาณ จึงปรับปรุงแผนงานมาใช้คำสั่ง friedmanchisquare จากไลบรารี Scipy ทดแทน

หนังสืออ้างอิง

1. Kenneth Sörensen, Marc Sevaux, and Fred Glover, “A History of Metaheuristics,” vol. to appear.
2. Sadegh Eskandari and Marjan Kuchaki Rafsanjani, “Two new selection methods and their effects on the performance of genetic algorithm in solving supply chain and travelling salesman problems,” vol. 22, no. 3, pp. 176–184.
3. Khalid Jebari, “Selection Methods for Genetic Algorithms,” vol. 3, pp. 333–344.
4. Ling Yu and Zu-yong Wan, “An Improved PSO Algorithm and Its Application to Structural Damage Detection,” in **2008 Fourth International Conference on Natural Computation**, vol. 1, pp. 423–427, ISSN: 2157-9563.
5. Jeffrey O. Agushaka, Absalom E. Ezugwu, Laith Abualigah, Samaher Khalaf Alharbi, and Hamiden Abd El-Wahed Khalifa, “Efficient Initialization Methods for Population-Based Metaheuristic Algorithms: A Comparative Study,” vol. 30, no. 3, pp. 1727–1787.
6. Raúl Martín-Santamaría, Manuel López-Ibáñez, Thomas Stützle, and J. Manuel Colmenar, “On the automatic generation of metaheuristic algorithms for combinatorial optimization problems,” vol. 318, no. 3, pp. 740–751.
7. Shih-Che Lo and Yi-Cheng Shih, “A Genetic Algorithm with Quantum Random Number Generator for Solving the Pollution-Routing Problem in Sustainable Logistics Management,” vol. 13, no. 15, pp. 8381, Number: 15 Publisher: Multidisciplinary Digital Publishing Institute.
8. Zahra Beheshti and Siti Mariyam Shamsuddin, “A review of population-based meta-heuristic algorithm,” vol. 5, pp. 1–35.
9. Kanchan Rajwar, Kusum Deep, and Swagatam Das, “An exhaustive review of the metaheuristic algorithms for search and optimization: taxonomy, applications, and open challenges,” vol. 56, no. 11, pp. 13187–13257.
10. S. Talukder, “Mathematical Modelling and Applications of Particle Swarm Optimization,” .
11. Ahmed G. Gad, “Particle Swarm Optimization Algorithm and Its Applications: A Systematic Review,” vol. 29, no. 5, pp. 2531–2561.
12. Sourabh Katoch, Sumit Singh Chauhan, and Vijay Kumar, “A review on genetic algorithm: past, present, and future,” vol. 80, no. 5, pp. 8091–8126.
13. Bushra Alhijawi and Arafat Awajan, “Genetic algorithms: theory, genetic operators, solutions, and applications,” vol. 17, no. 3, pp. 1245–1256.
14. Anand Nayyar, Vikram Puri, and G. Suseendran, “Artificial Bee Colony Optimization—Population-Based Meta-Heuristic Swarm Intelligence Technique,” in **Data Management, Analytics and Innovation**, Valentina Emilia Balas, Neha Sharma, and Amlan Chakrabarti, Eds. pp. 513–525, Springer.
15. Vaishali R Kulkarni and Veena Desai, “ABC and PSO: A comparative analysis,” in **2016 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC)**, pp. 1–7, ISSN: 2473-943X.
16. Makoto Matsumoto and Takuji Nishimura, “Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator,” vol. 8, no. 1, pp. 3–30.
17. William J. Morokoff and Russel E. Caflisch, “Quasi-Random Sequences and Their Discrepancies,” vol. 15, no. 6, pp. 1251–1279, Publisher: Society for Industrial and Applied Mathematics.
18. Engin Afacan, Gonenc Berkol, G. Dundar, and Faik Başkaya, “An Analog Circuit Synthesis Tool based on Efficient and Reliable Yield Estimation,” vol. 54.
19. D. J. Shepherd, “On the Role of Hadamard Gates in Quantum Circuits,” vol. 5, no. 3, pp. 161–177.
20. Anastasios Bikos, Panagiotis E. Nastou, Georgios Petroudis, and Yannis C. Stamatiou, “Random Number Generators: Principles and Applications,” vol. 7, no. 4, pp. 54, Number: 4 Publisher: Multidisciplinary Digital Publishing Institute.
21. Nguyen Van Thieu, “Opfunu: An Open-source Python Library for Optimization Benchmark Functions,” vol. 12.
22. Sathya Nagarajan and A. Muthukumaravel, “A Review of the Optimization Algorithms on Traveling Salesman Problem,” vol. 8.
23. Gerhard Reinelt, “TSPLIB,” .
24. Andrew Rukhin, Juan Soto, James Nechvatal, Elaine Barker, Stefan Leigh, Mark Levenson, David Banks, Alan Heckert, and James Dray, “A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications,” .

25. Erick Cantu-Paz, “On Random Numbers and the Performance of Genetic Algorithms,” .
26. Jeffrey O. Agushaka and Absalom E. Ezugwu, “Initialisation Approaches for Population-Based Metaheuristic Algorithms: A Comprehensive Review,” vol. 12, no. 2, pp. 896, Number: 2 Publisher: Multidisciplinary Digital Publishing Institute.
27. “ANU QRNG – Quantum random numbers,” .
28. Xiongfeng Ma, Xiao Yuan, Zhu Cao, Bing Qi, and Zhen Zhang, “Quantum random number generation,” vol. 2, no. 1, pp. 1–9, Publisher: Nature Publishing Group.
29. X. Wang and F. J. Hickernell, “Randomized Halton sequences,” vol. 32, no. 7, pp. 887–899.
30. Qian Li, San-Yang Liu, and Xin-She Yang, “Influence of initialization on the performance of metaheuristic optimizers,” vol. 91, pp. 106193.
31. Zhengbing Hu, Sergiy Gnatyuk, Tetiana Okhrimenko, Sakhybay Tynymbayev, and Maksim Iavich, “High-Speed and Secure PRNG for Cryptographic Applications,” vol. 12, no. 3, pp. 1.
32. Luca Crocetti, Stefano Di Matteo, Pietro Nannipieri, Luca Fanucci, and Sergio Saponara, “Design and Test of an Integrated Random Number Generator with All-Digital Entropy Source,” vol. 24, no. 2, pp. 139, Number: 2 Publisher: Multidisciplinary Digital Publishing Institute.